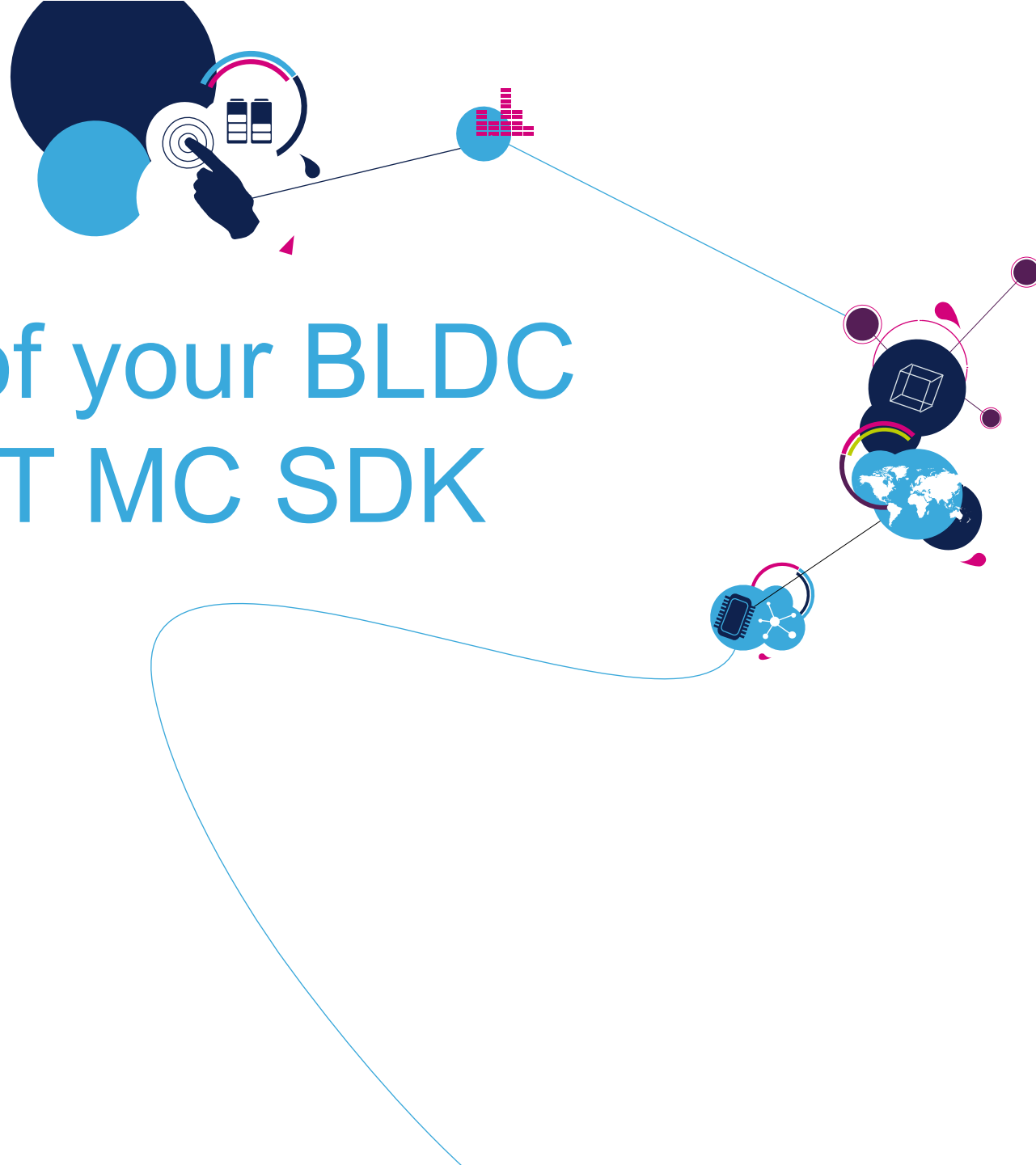




Troubleshooting of your BLDC motor design in ST MC SDK

Tomas DRESLER
Ondrej HOLY
Tadeas HOLLER

Time

Presentation

0:00

Why troubleshooting?

0:15

Error solving in MC Workbench

0:30

Prerequisites

0:45

What do you need to validate?

1:15

Tips for solving runtime issues

1:45

Tips for FOC start-up

2:00

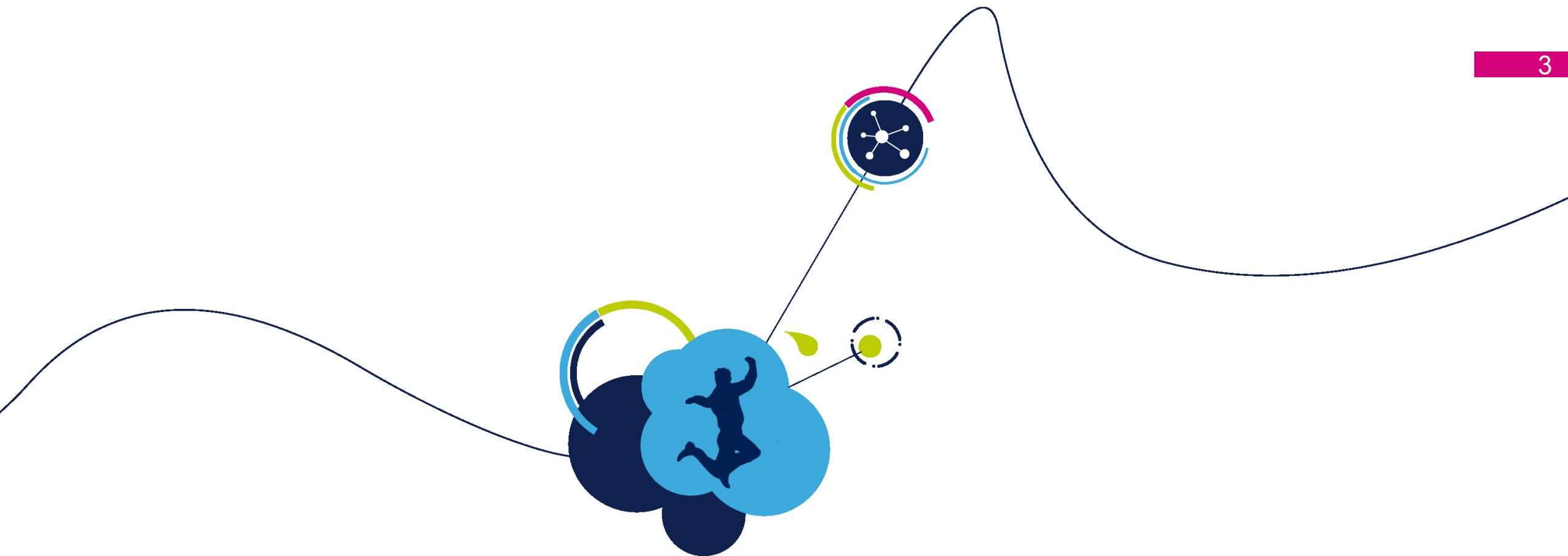
Tips for tuning PI regulators in MC workbench

2:15

General and analytic PI regulator tuning

2:45

Development of own hardware setup



Why troubleshooting?

Why troubleshooting?

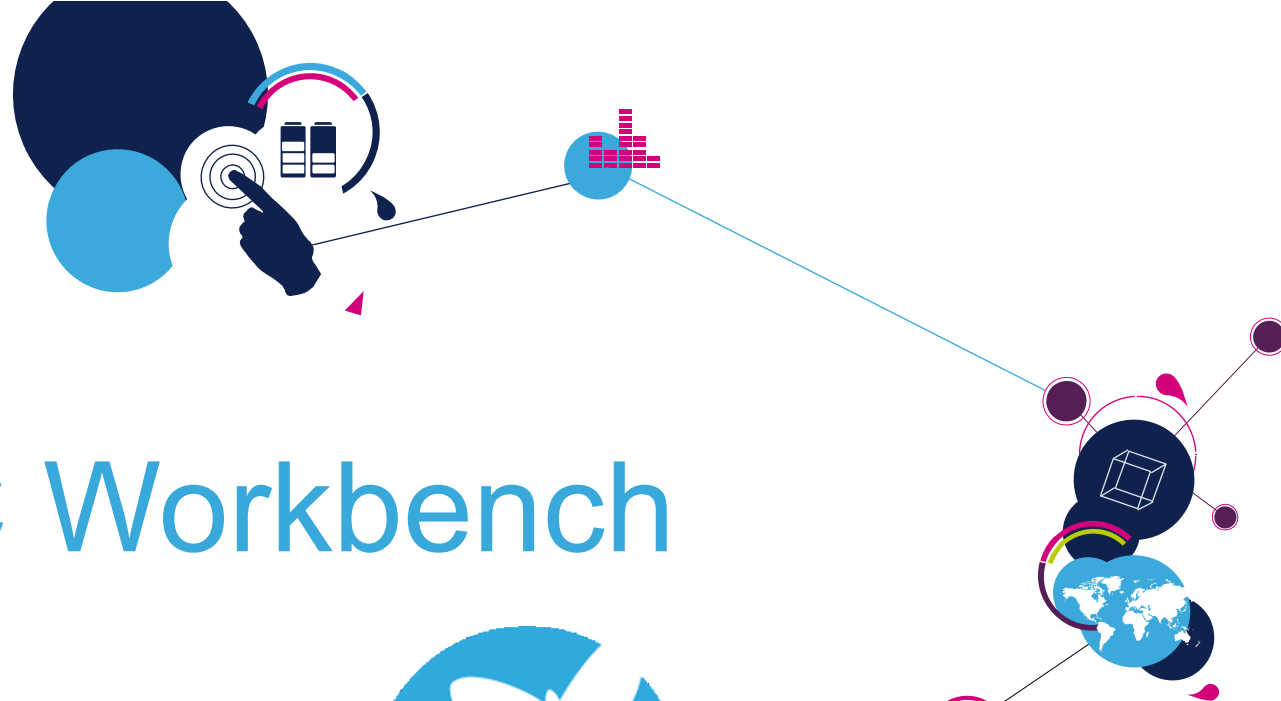
4

- Each setup is different:
 - Designer has to adapt MC SDK settings to new hardware, motor, application requirements and control strategies
 - Some setup combinations may induce limitations in the generated code
- Each motor application is different:
 - Load, changing over time
 - Electrical parameters differ from application to application
 - The assumptions in the SDK target mainstream engines, not special ones
 - i.e., very low inductance windings (high-speed motors)
 - Very high load variations (no-load vs. heavy load on the clutch)

Why troubleshooting?

5

- Every designer was a beginner
 - And made some assumptions that were not right,
 - Or omitted some crucial detail,
 - Or reused old design without checking the setup too much etc.
- Every library comes with some limitations
 - Known or unknown, developer has to tackle these to bring his project to the proper end!



Error solving in MC Workbench



We have an error in MC Workbench!

7

The screenshot shows the ST Motor Control Workbench interface. A red box highlights the 'Control Stage' icon in the left sidebar, with a red arrow pointing to it and the text 'Double click'. Another red box highlights the 'Generation failed!' error dialog box, which says 'See log bar for more information.' and has an 'OK' button. A third red box highlights the error message in the log bar: 'Maximum resulting modulation index clamped to minimum allowed (81%)'. The log bar also shows the time '02:19:04' and the motor ID '1'. The main workspace shows a block diagram of the 'Control Unit' with sub-blocks for 'MCU and Clock Frequency', 'Analog Input and Protection', 'DAC functionality', and 'Digital I/O', connected to a 'User Interface' block. The ST logo and 'life.augmented' text are visible in the bottom right corner.

ST Motor Control Workbench [MC_WorkShop_01]

File Tools Help Documentation

Control Stage

Motor

Power Stage

Drive Management

MCU and Clock Frequency

Control Unit

Analog Input and Protection

DAC functionality

Digital I/O

User Interface

NUCLEO-F303RE - custom

ST Motor Control Workbench

Generation failed!
See log bar for more information.

OK

Double click

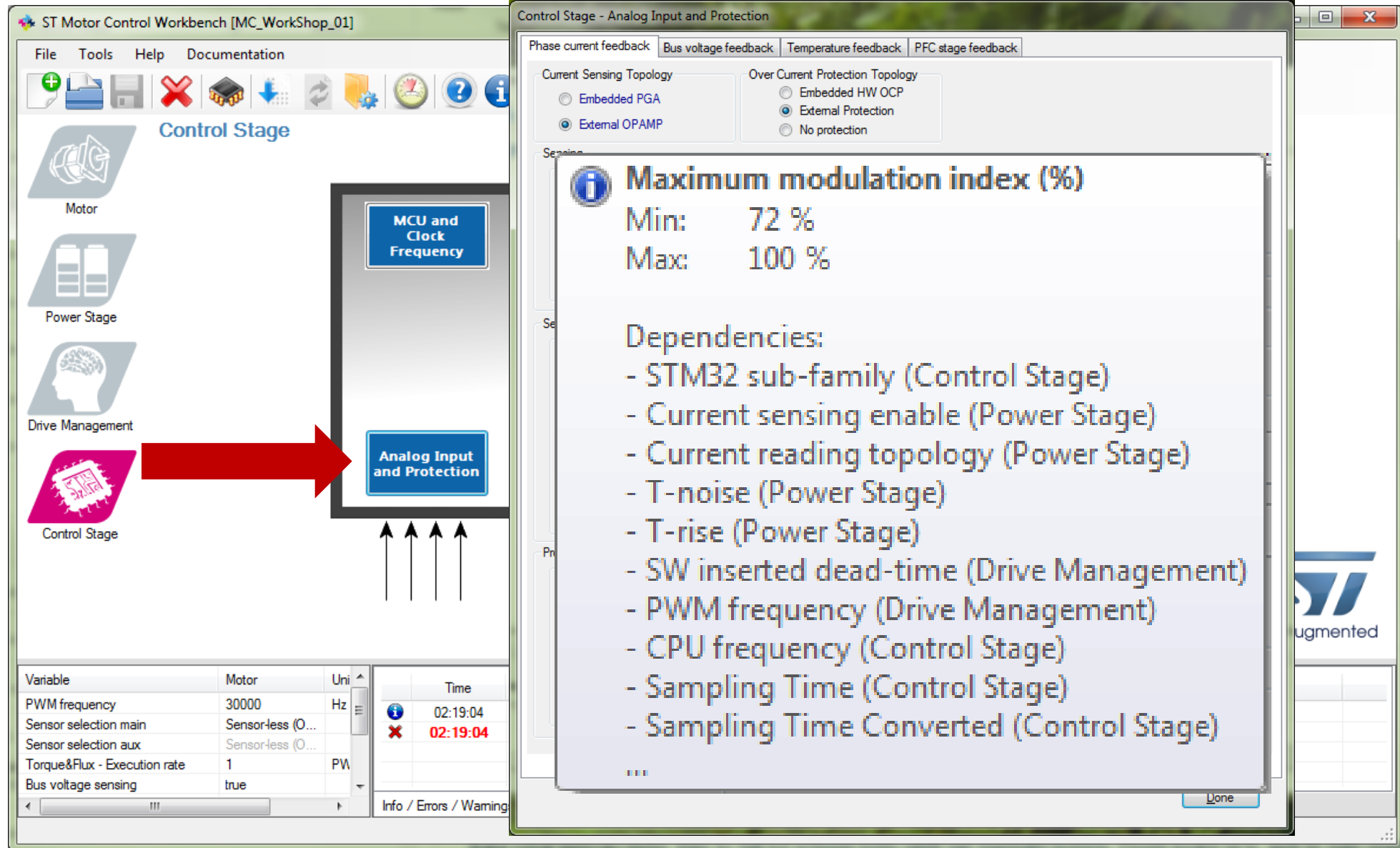
Variable	Motor	Unit	Time	Motor	Id	Message
PWM frequency	30000	Hz	02:19:04	1		Maximum resulting modulation index clamped to minimum allowed (81%)
Sensor selection main	Sensor-less (O...					
Sensor selection aux	Sensor-less (O...					
Torque&Flux - Execution rate	1	PW				
Bus voltage sensing	true					

Info / Errors / Warnings Change Log

ST life.augmented

How can we solve the problem?

8



The screenshot displays the ST Motor Control Workbench [MC_WorkShop_01] interface. On the left, the 'Control Stage' is highlighted in the project tree, with a red arrow pointing to the 'Analog Input and Protection' block in the main workspace. The workspace also shows 'MCU and Clock Frequency' and 'Power Stage' blocks. A table at the bottom left lists variables for the Motor, and a status bar at the bottom right shows a warning at 02:19:04.

Control Stage - Analog Input and Protection

Phase current feedback | Bus voltage feedback | Temperature feedback | PFC stage feedback

Current Sensing Topology: ☐ Embedded PGA, ☒ External OPAMP

Over Current Protection Topology: ☐ Embedded HW OCP, ☒ External Protection, ☐ No protection

Maximum modulation index (%)

Min: 72 %
Max: 100 %

Dependencies:

- STM32 sub-family (Control Stage)
- Current sensing enable (Power Stage)
- Current reading topology (Power Stage)
- T-noise (Power Stage)
- T-rise (Power Stage)
- SW inserted dead-time (Drive Management)
- PWM frequency (Drive Management)
- CPU frequency (Control Stage)
- Sampling Time (Control Stage)
- Sampling Time Converted (Control Stage)

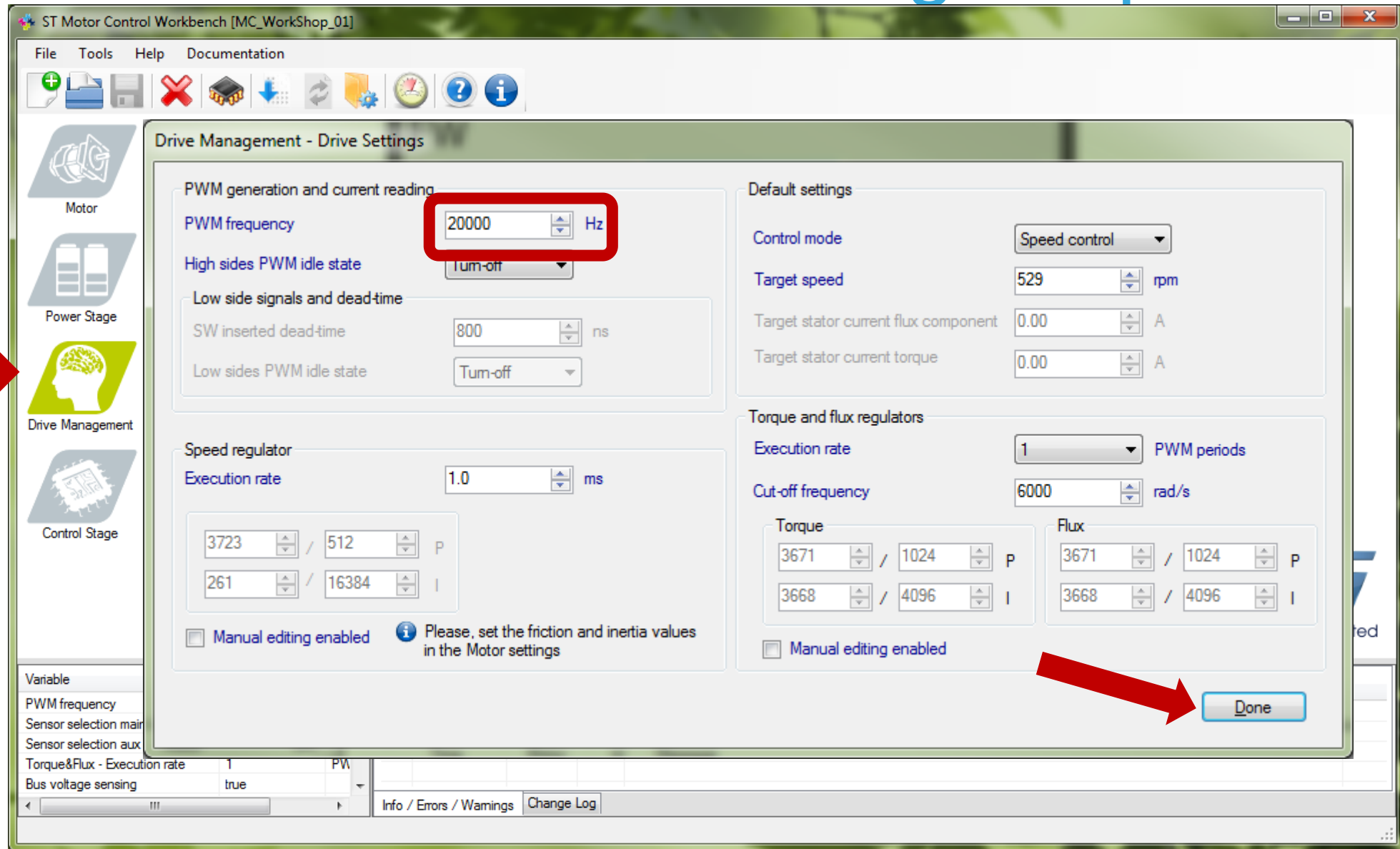
Variable	Motor	Unit
PWM frequency	30000	Hz
Sensor selection main	Sensor-less (O...	
Sensor selection aux	Sensor-less (O...	
Torque&Flux - Execution rate	1	PW
Bus voltage sensing	true	

Time: 02:19:04 (Warning icon)

Info / Errors / Warning

Solving the problem

9



Error solved – project save and regeneration

10

ST Motor Control Workbench [MC_WorkShop_01]*

File Tools Help Documentation

Control Stage

Motor

Power Stage

Drive Management

Control Stage

Project generation ongoing...

Version info:

- MC Workbench : 5.0.2.18126
- WB_to_Mx : 0.9.6.template-18-03-22
- STM32CubeMX : 4.25.0
- MC Firmware Library: 5.0.2

Generating...

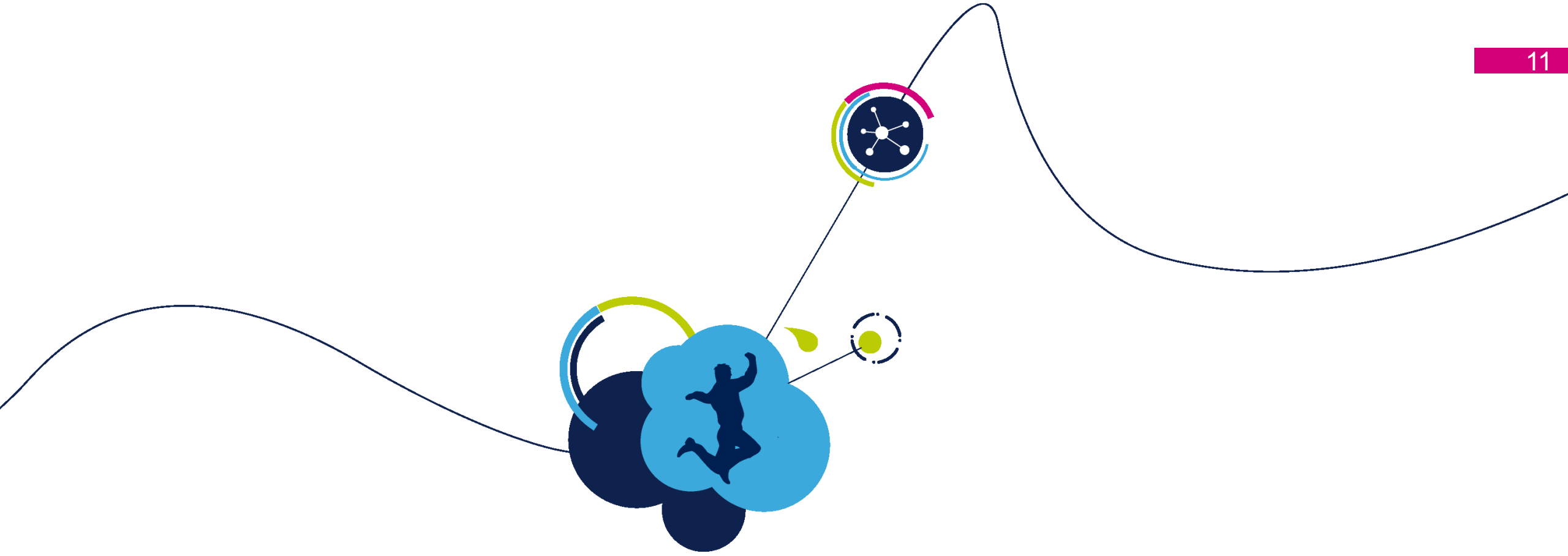
D:\2018_Q2_MotorControlWorkshop\Working_HandsOn\MC_WorkShop_01\
... \MC_WorkShop_01.ioc
... \MC_WorkShop_01.ioc.bak
... \extSettings
Code generation started

life.augmented

Variable	Motor	Uni	Time	Motor	Id	Message
			01:41:56			Generation files starting
			01:42:26			Project files generated on folder:'D:\2018_Q2_MotorControlWorkshop\Working_HandsOn\MC_WorkShop_01'

Info / Errors / Warnings Change Log

life.augmented



Prerequisites

Prerequisites

- What do I need to successfully debug my motor application?
 - Multichannel Digital Signal Oscilloscope
 - with current probe
 - and several (high)-voltage probes
 - Insulation transformer(s) (for high-voltage applications)
 - Multimeter
- What else?
 - Board schematics
 - DAC outputs
 - Debugger (possibly insulated)
 - Serial to USB cable (possibly insulated)

- DAC outputs

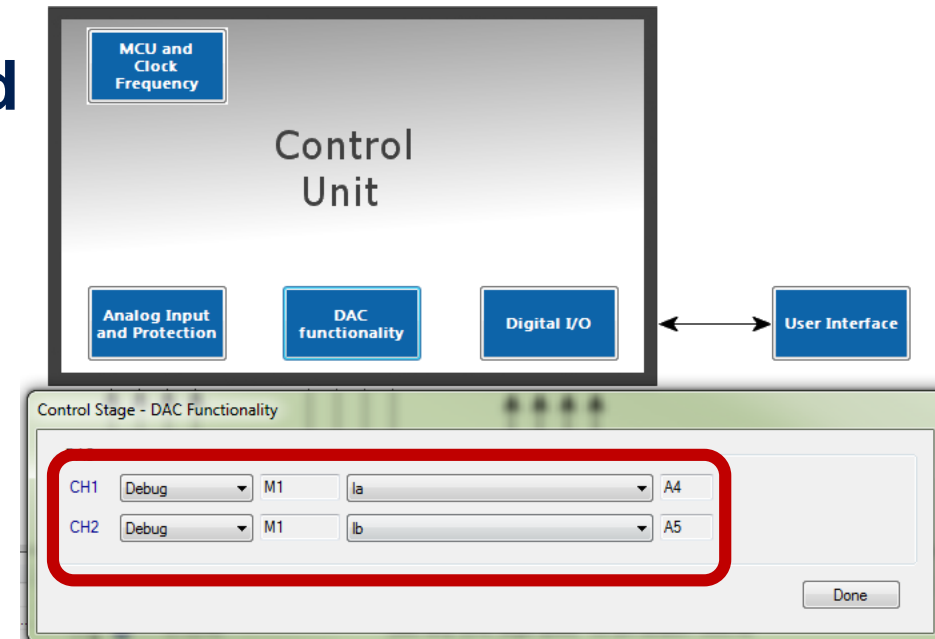
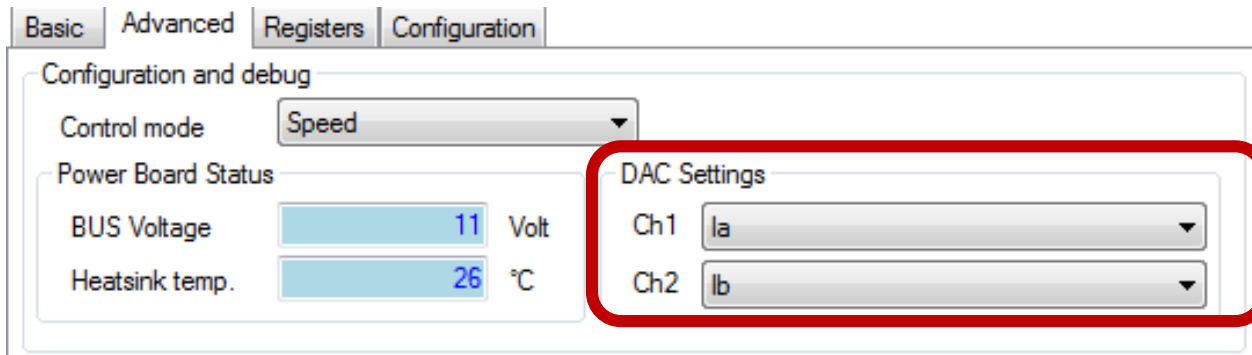
- Display up to 2 run-time values of variables used in the FOC algorithm using oscilloscope
- Allow immediate look at internal processes
- I_{α} , I_{β} show quality of current reading – if noisy or distorted, validate current reading path and setup t_{noise} , t_{raise} according to FOC SDK User Manual
- $b\text{-emf}_{\alpha}$, $b\text{-emf}_{\beta}$, observed el. angle – if noisy or unstable, $G2$ is too high or measurement of R , L , K_e of your motor are wrong (within range of tens of %)
- Options for DAC output supported by MC FOC SDK:

DAC	PWM	SPI
Must be supported by MCU and your HW Very precise	Needs RC filter Cheap, output has exponential delay	Requires external DAC Very precise

Prerequisites – DAC channels

14

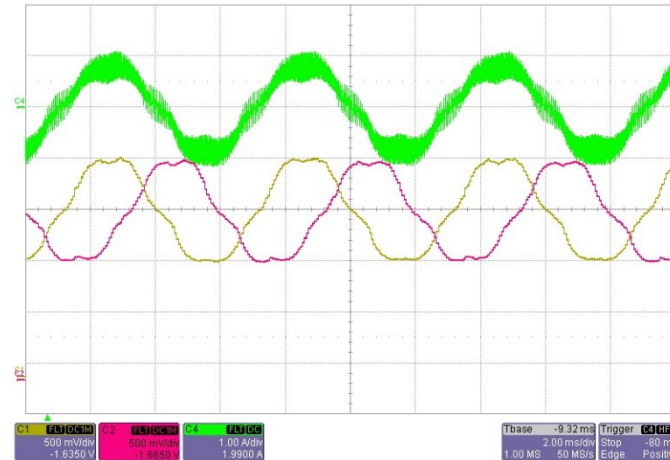
- DAC functionality can help to debug and tune the application.
 - CH1 – pin PA4, CH2 – pin PA5
 - Availability depends on choice of STM32 family
- Enable & Select initial outputs in the **DAC functionality**:
- Select output in the **Monitor** – tab **Advanced**



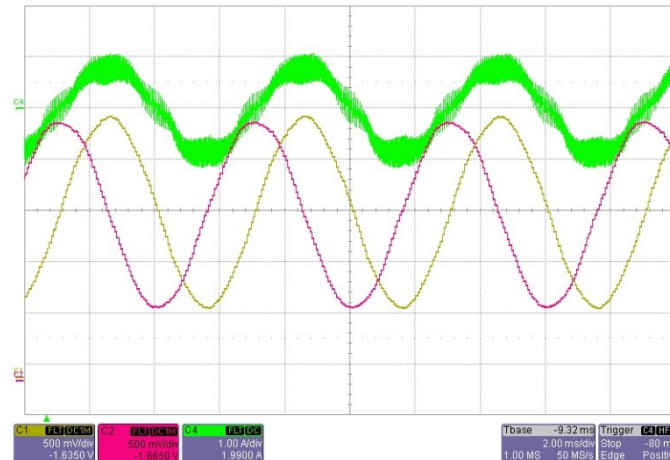
Prerequisites – DAC channels

15

- Typical DAC waveforms of tuned system



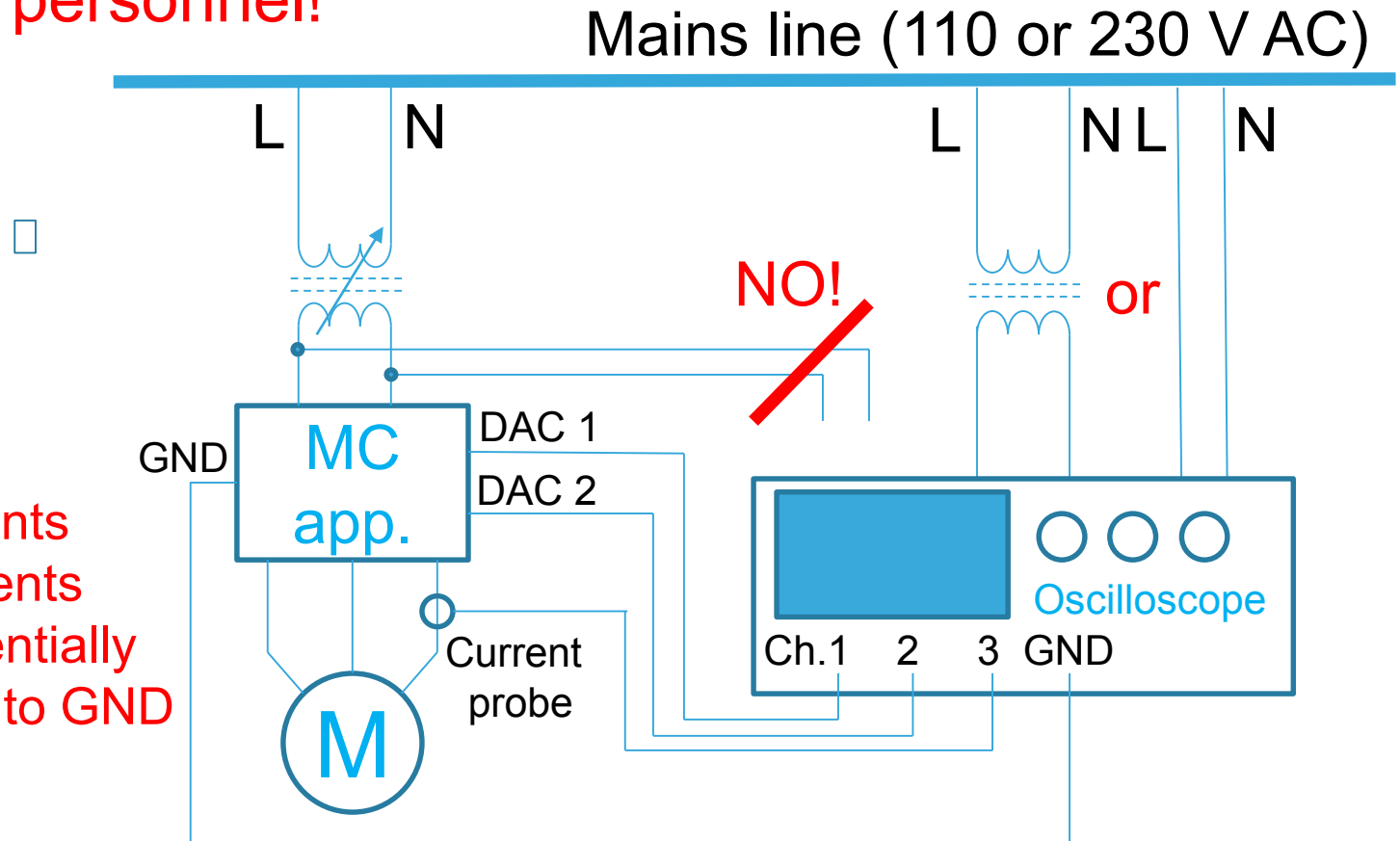
- Green: phase A motor current
- Yellow: DAC ch1 (Ia)
- Pink: DAC ch2 (Ib)

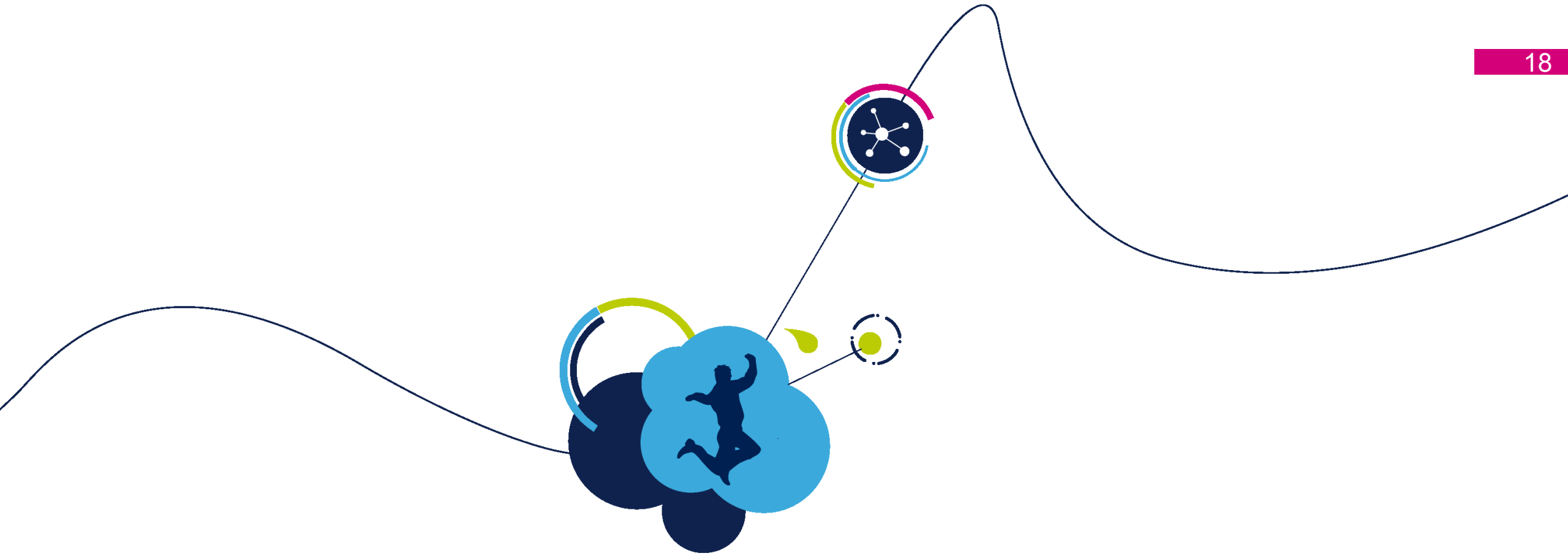


- Green: phase A motor current
- Yellow: DAC ch1 (Obs. BEMF Alpha)
- Pink: DAC ch2 (Obs. BEMF Beta)

- UART (via USB) connection allows natively
 - debugging with MC Workbench in Monitor mode,
 - changing runtime parameters (coefficients of many PI regulators, G2, required speed or torque etc.),
 - observing speed and tuning PI coefficients of speed regulator in runtime
- Nucleo boards have virtual COM port emulated by ST-Link, which is connected to STM32 UART and allows connection of various tools
 - The Nucleo's ST-Link can be split from the STM32 and used as standalone debugger and/or virtual serial port, if needed

- Insulation transformer(s), oscilloscope usage in high-voltage systems
- **Operate only by certified personnel!**
 - Reduce risk of equipment damage
 - Reduce risk of injury or death
 - For mains (AC) rated applications
 - Oscilloscope isolation prevents from damage if leaking currents are present in MC app. Potentially high voltage can be applied to GND

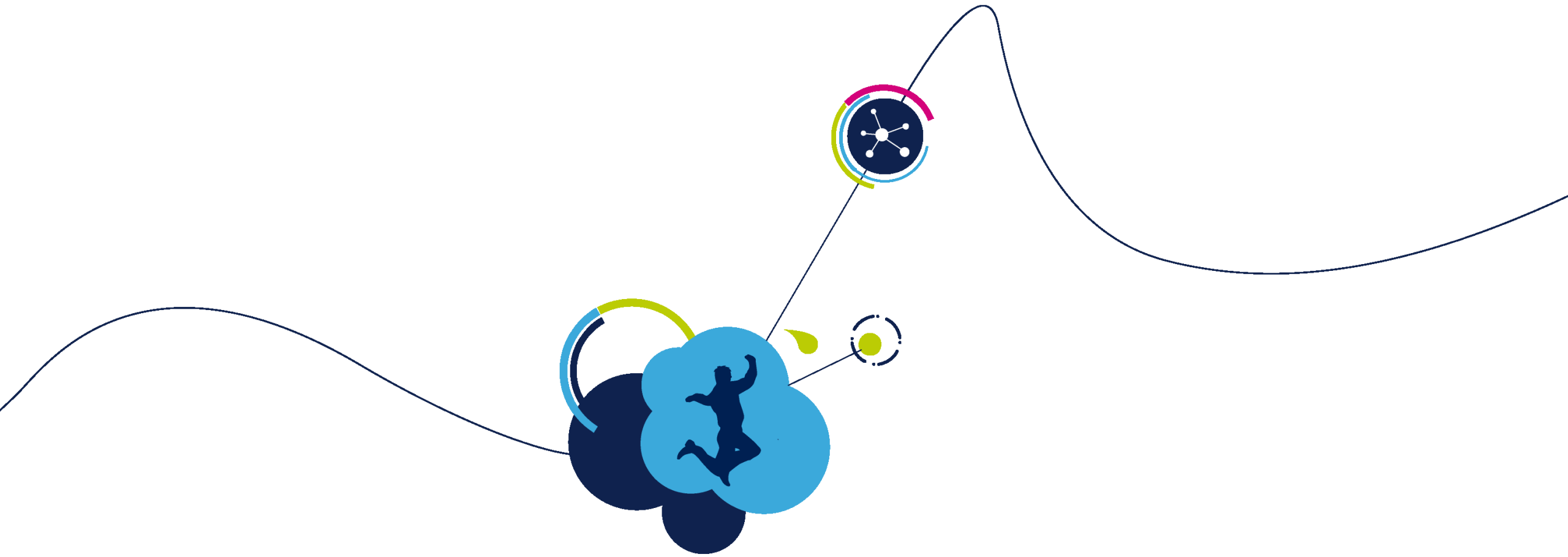




What do you need to validate?

- Before I switch the application on
 - Proper jumper setup and soldering switch configuration for:
 - Current reading topology (1-shunt, 3-shunt)
 - Speed sensor configuration (Hall, encoder, its derived supply: 5V/3.3V)
 - Overcurrent, over-temperature reaction mechanism
 - Polarity and rating of supply voltage, setup and range of DC-DC converter
 - Is isolation needed between mains and your application and your oscilloscope?
 - Does my power supply have current limiting option?
Initially set to small value (tens of mA)!
 - Are my DAC outputs connected to the oscilloscope?

- After I switch the application on
 - Is the supply current within a reasonable limit?
Without motor running, only few tens of mA are suggested.
Only after this check, set current limitation to operating limit!
 - Is your application alive and communicating (via UI, USART etc.)?
 - When I start the motor, is the overcurrent reported by power supply or by overcurrent mechanism of the power stage?
 - Do I see the currents reported by DAC outputs on my oscilloscope when my motor runs?



Tips for solving runtime issues

Tips for solving runtime issues

22

- Initial application engineer's decision tree:
 1. If it burns, check HW
 2. If it doesn't move, check power stage setup (PWM, ADC, OC)
 3. If it moves a little, check alignment, start-up and rev-up
 4. If it moves, but fails at speed change, check G2 and Speed PI regulator
 5. If it moves, but speed is unstable, check Speed PI regulator
 6. If it moves well, you're finished! Congratulations!
- Application design
 1. Does it need speed control? Use torque mode!
 2. Does it really need speed control? Use speed mode, but tune Speed PI well!

Tips for solving runtime issues

23

- I don't see changes of my SW in the application
 - Did you generate updated header files in correct folder or placed them there?
 - Did you recompile your SW after applying the changes?
 - Did you load new SW to your application?
 - Isn't the debugger the other USB cable connected to your PC?

Tips for solving runtime issues

24

- Does your application read, evaluate and react on SDK error states?
 - If not, rethink your SW and ask the question again!
- “FOC duration” error shows up, motor doesn’t move at all
 - The PWM frequency is too high for CPU to handle FOC algorithm in time
 - Increase “FOC execution rate” by one in Drive settings
 - Revalidate your IRQ priorities – MC FOC SDK must always have the highest interrupt priority of all! Your motor depends on it!

Tips for solving runtime issues

25

- Best PWM frequency?
 - Low enough to reduce switching losses (down to ~8 kHz)
 - High enough to manage low inductance motors (up to ~30 kHz) and above acoustic noise band (human ear can listen up to 22 kHz)
 - CPU load can be reduced by increasing “FOC execution rate”
- Best start-up settings (start with, tune to your application later)
 - Speed ramp duration 3000 ms
 - Speed ramp final value 30% of maximum motor speed
 - Current ramp final value 50% of nominal motor current
 - Include alignment (2000 ms, 50% of nominal motor current)
 - Minimum start-up speed set as 15% of maximum motor speed

Tips for solving runtime issues

26

- “Over current” detected immediately at start-up
 - Wrong current sensing topology – single-shunt instead of three-shunt? Did you check your jumpers?
 - Isn't power stage damaged by overcurrent? Check your MOSFETs for short circuit!
 - Verify and correct current path gain, ADC input, polarity of PWM outputs
 - The current regulation bandwidth is too high – in Drive settings, reduce current regulation bandwidth down to 2000 rad/s for 3-shunt and 1000 rad/s for 1-shunt. Maximum value is 9000 rad/s, resp. 4500 rad/s.
 - PWM frequency is too low – phase current may rise too quickly esp. in low-inductance motors – increase PWM frequency, to avoid “FOC duration” error see previous slide

Tips for solving runtime issues

27

- “Speed feedback” appears after sudden acceleration or deceleration

The runtime uses speed variation for detecting anomalies, thus big speed change over short time can trigger this error

- Use speed ramps to reduce acceleration
- Observer gain G2 is too high and speed estimation generates too much noise:
 - Decrease G2 by 2, 4, 6, 8 and retry...
 - Detect by displaying b-emf alpha and beta on the DACs – they shall be sinusoidal. Too much noise or distortion means too high G2 estimation
- Test in torque mode. If this mode runs stable, speed PI regulator needs re-tuning
- Too low variance threshold. Increase gradually up to 100% or increase speed FIFO depth (as power of 2).
Beware, this parameter significantly reduces sensitivity for rotor locked condition detection!

Tips for solving runtime issues

28

- “Speed feedback” appears immediately after start-up
 - Start-up validity can be detected too early. Return to Start-up parameters and increase # of successful consecutive start-up tests (not more than 4-5) and minimum start-up speed to 15% of maximum motor speed
 - Use Rev-up algorithm to align gradually forced torque ramp with sensor-less control algorithm output – this avoids sudden speed change at the transition between open loop and closed loop
- “Start-up failure” appears after motor initially moves, but stops before Rev-up
 - The torque (current) at higher start-up speed is insufficient to accelerate to validation speed
 - Decrease acceleration rate – set speed ramp final value to 30% of motor max. speed
 - Increase start-up current – from ~50% up to 100% of nominal current
 - Enable alignment phase for more deterministic start-up or use advanced start-up

Tips for solving runtime issues

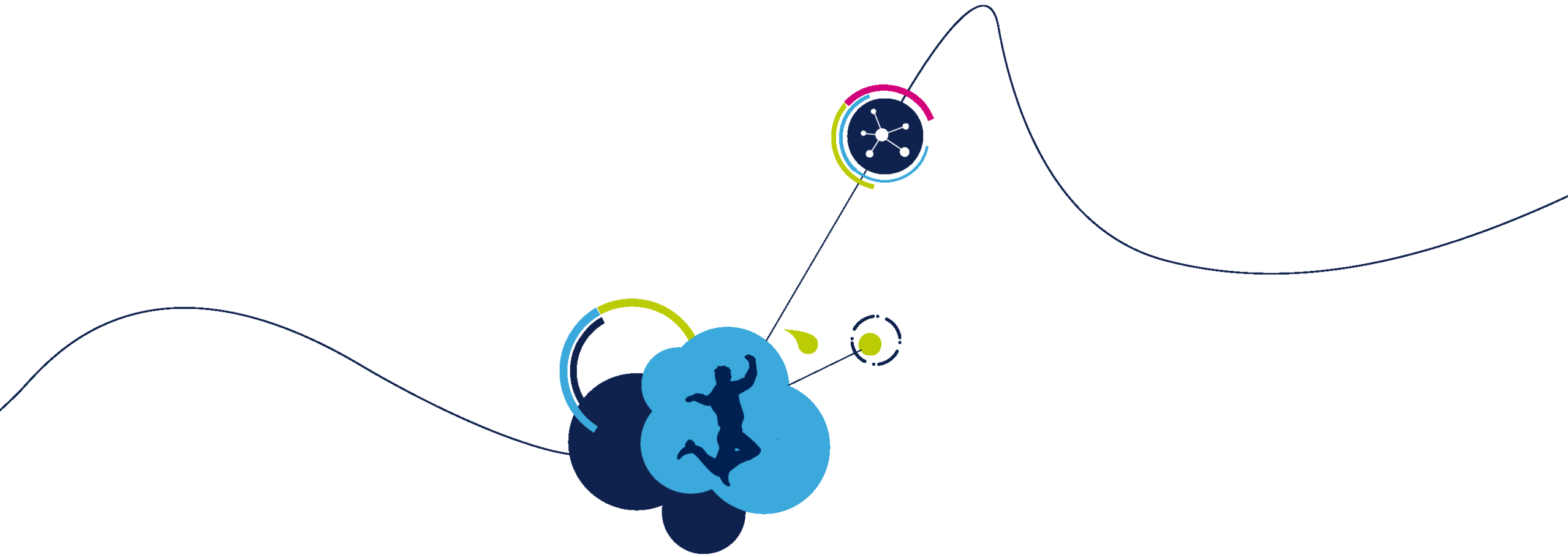
29

- My motor doesn't start immediately
 - You have opted for alignment – motor waits for stabilization in specific position
 - “On-the-fly start-up” has been selected – detection phase has started and waits some time for rotation signal – if none is found, standard alignment and start-up applies

Tips for solving runtime issues

30

- Speed or control is unstable below 5% of maximal motor speed
 - The sensorless algorithm is stable down to 5-7% of maximum speed. Below that speed the noise from current reading is too high for proper rotor position estimation.
 - Solution is twofold:
 1. Use Hall sensors for low speed region and sensorless at medium-to-high speeds
 2. Use HFI sensorless algorithm (only available for anisotropic motors and MCUs with FPU)
- Speed control is unstable at various speeds
 - Load changes in non-linear fashion
 - May be too light at low and high speeds (i.e. air pump) and heavy at medium speeds
 - May require different Speed PI tuning at different speeds with linear interpolation between the regions



Tuning FOC start-up

Basic and advanced methods

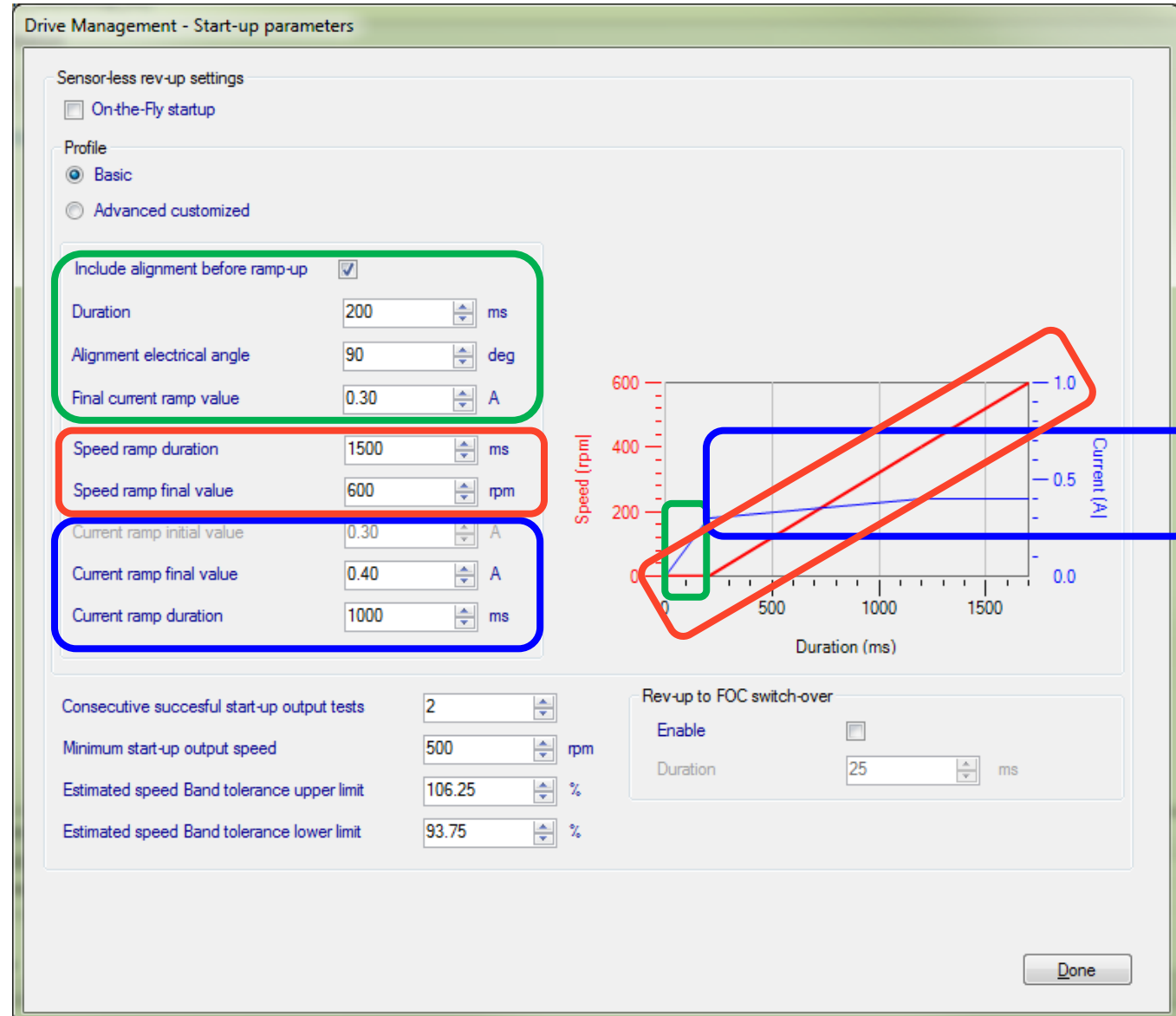
FOC StartUp - Basic

32

- Duration depends on construction of motor - the higher mass, the higher duration. Initially, try 2000 ms
- Final current ramp value from 0.5 to 1 times motor nominal current according to load

- Set speed ramp final value to around 30% of maximum application speed
- According to motor inertia it may be required to increase the speed ramp duration

- Set current ramp initial and final values equal to motor nominal current value / 2



FOC StartUp - Basic

33

- Set minimum start-up output speed to 15% of maximum application speed (if required, decrease it later)
- Set estimated speed band tolerance lower limit to 93.75%

Drive Management - Start-up parameters

Sensor-less rev-up settings

☐ On-the-Fly startup

Profile

☒ Basic

☐ Advanced customized

Include alignment before ramp-up ☒

Duration 200 ms

Alignment electrical angle 90 deg

Final current ramp value 0.30 A

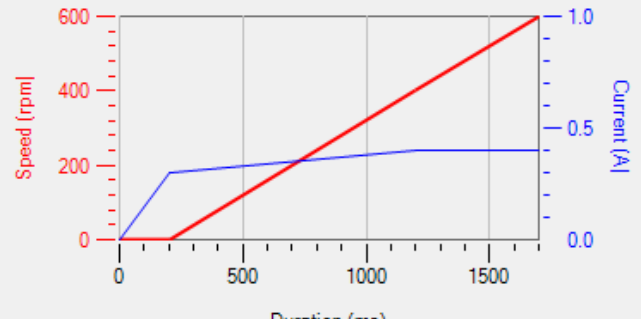
Speed ramp duration 1500 ms

Speed ramp final value 600 rpm

Current ramp initial value 0.30 A

Current ramp final value 0.40 A

Current ramp duration 1000 ms



Consecutive successful start-up output tests 2

Minimum start-up output speed 500 rpm

Estimated speed Band tolerance upper limit 106.25 %

Estimated speed Band tolerance lower limit 93.75 %

Rev-up to FOC switch-over

Enable ☐

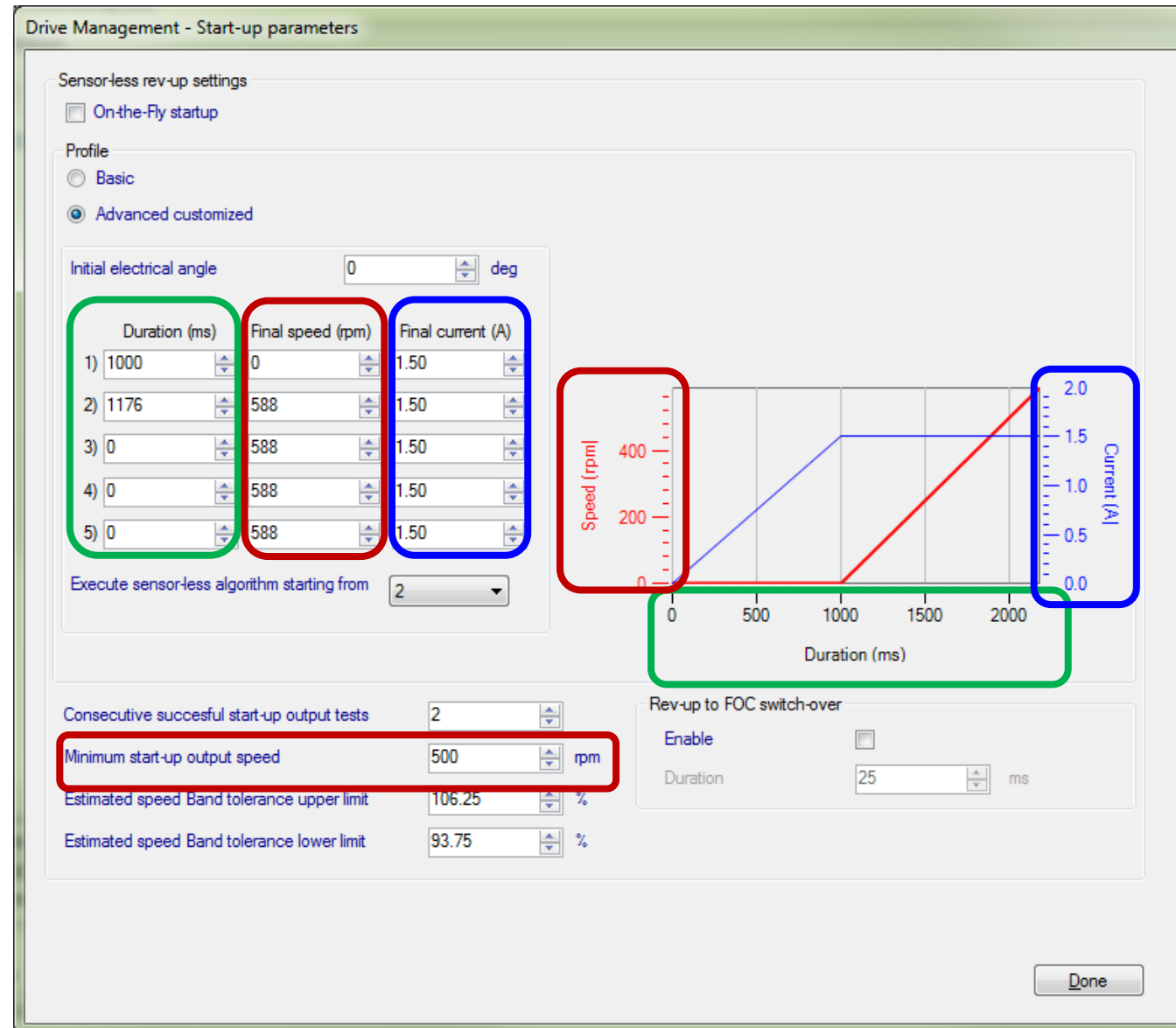
Duration 25 ms

Done

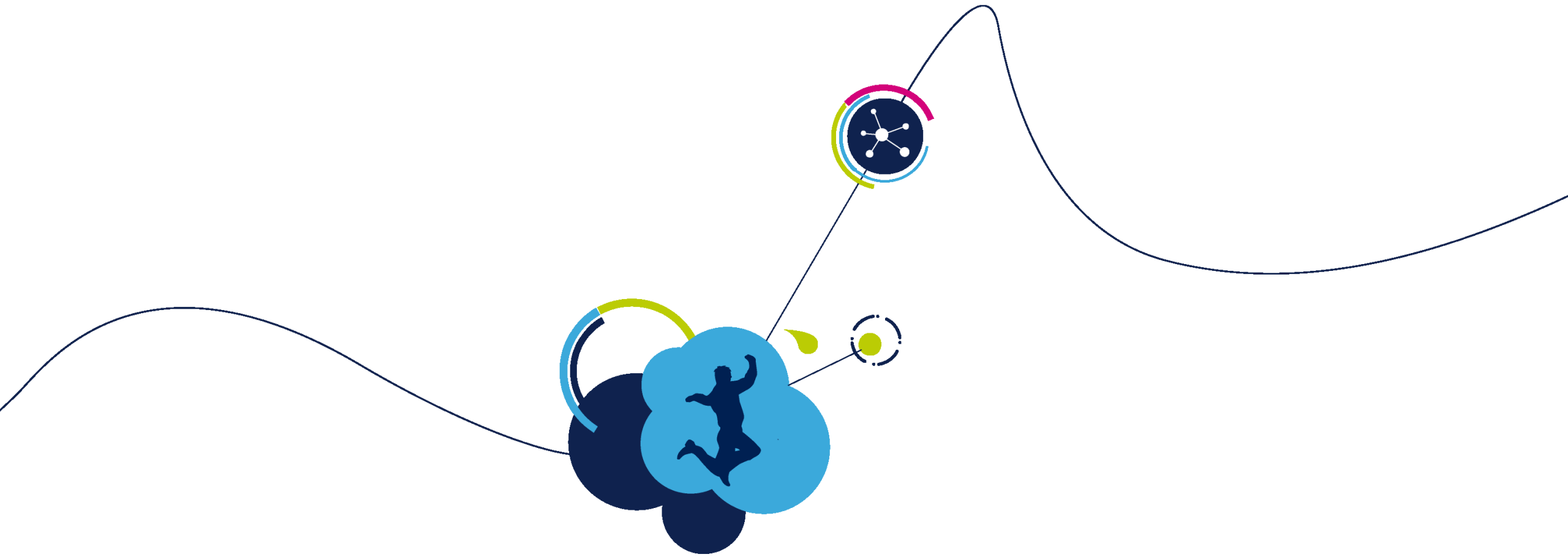
FOC StartUp - Advanced

34

- Variable setting of start ramp by steps



Threshold for to switch to the close loop!

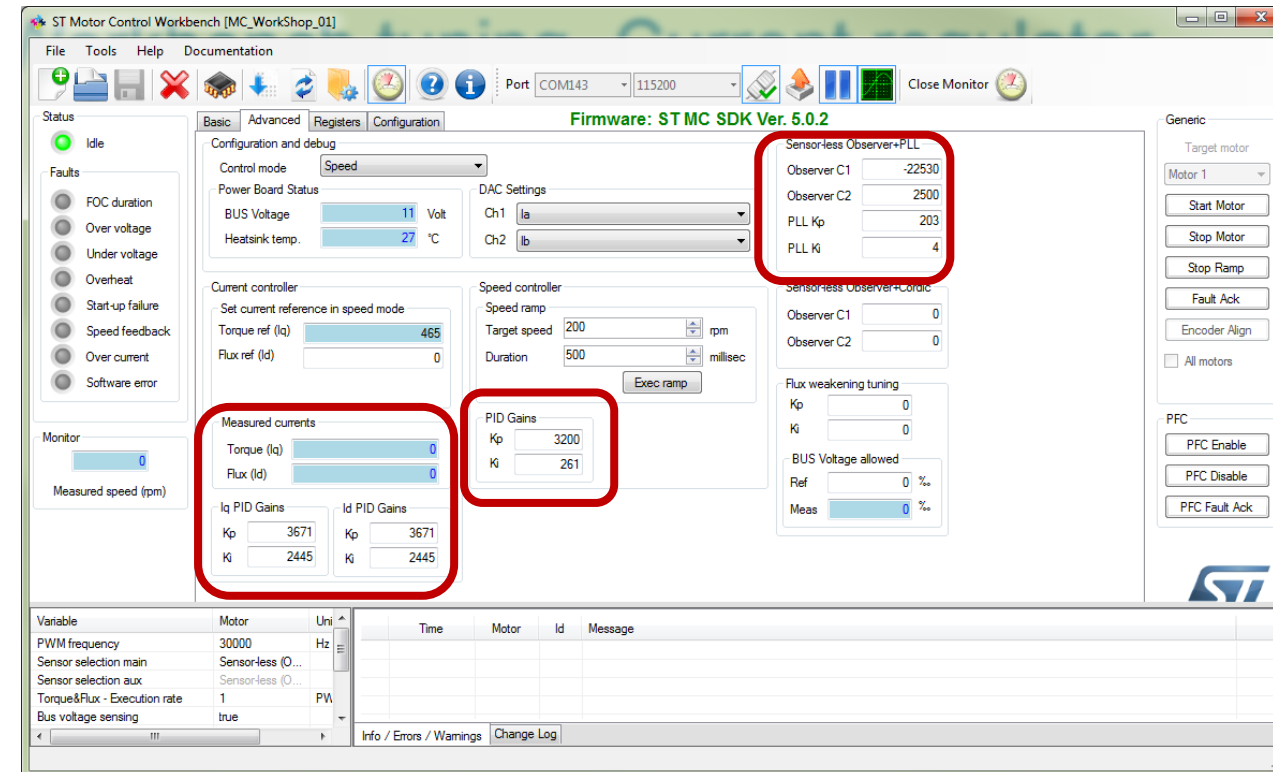


Tips for tuning PI regulators in MC SDK
in the MC Workbench

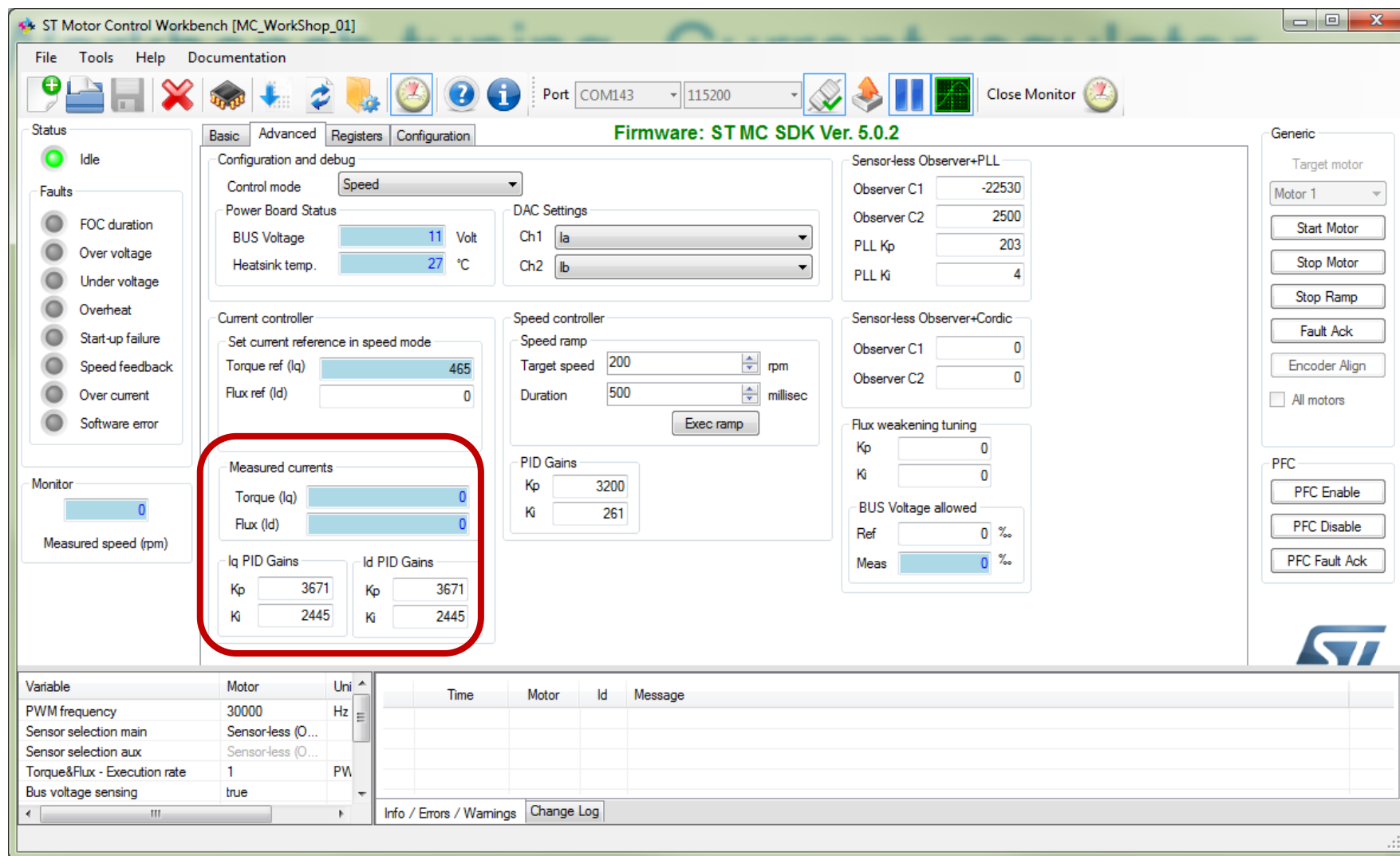
Tuning sequence

37

- We recommend this sequence of steps for tuning the PID parameters
 1. Current regulator
 2. Observer (C2) and Observer regulator
 - In Torque mode skip the Speed regulator
 3. Speed regulator



MC Workbench tuning - Current regulator 38



MC Workbench tuning - Current regulator

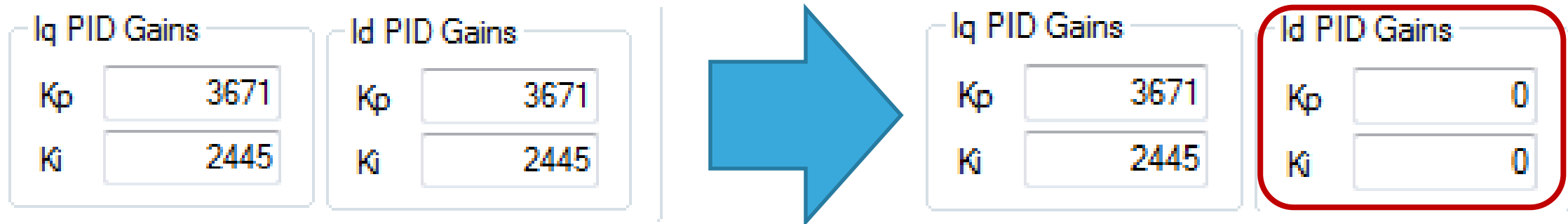
39

- How can you check whether the current regulator is set properly?
- What do we need?
 - DAC output
 - Oscilloscope
 - Current probe
- The rotor has to be fixed (by hand, by screw, etc.)
- We have to apply a unit-pulse current signal and observe curve of a current
- How can we do it?

MC Workbench tuning - Current regulator

40

- Set **K_p** and **K_i** for **I_d** PID Gains to 0



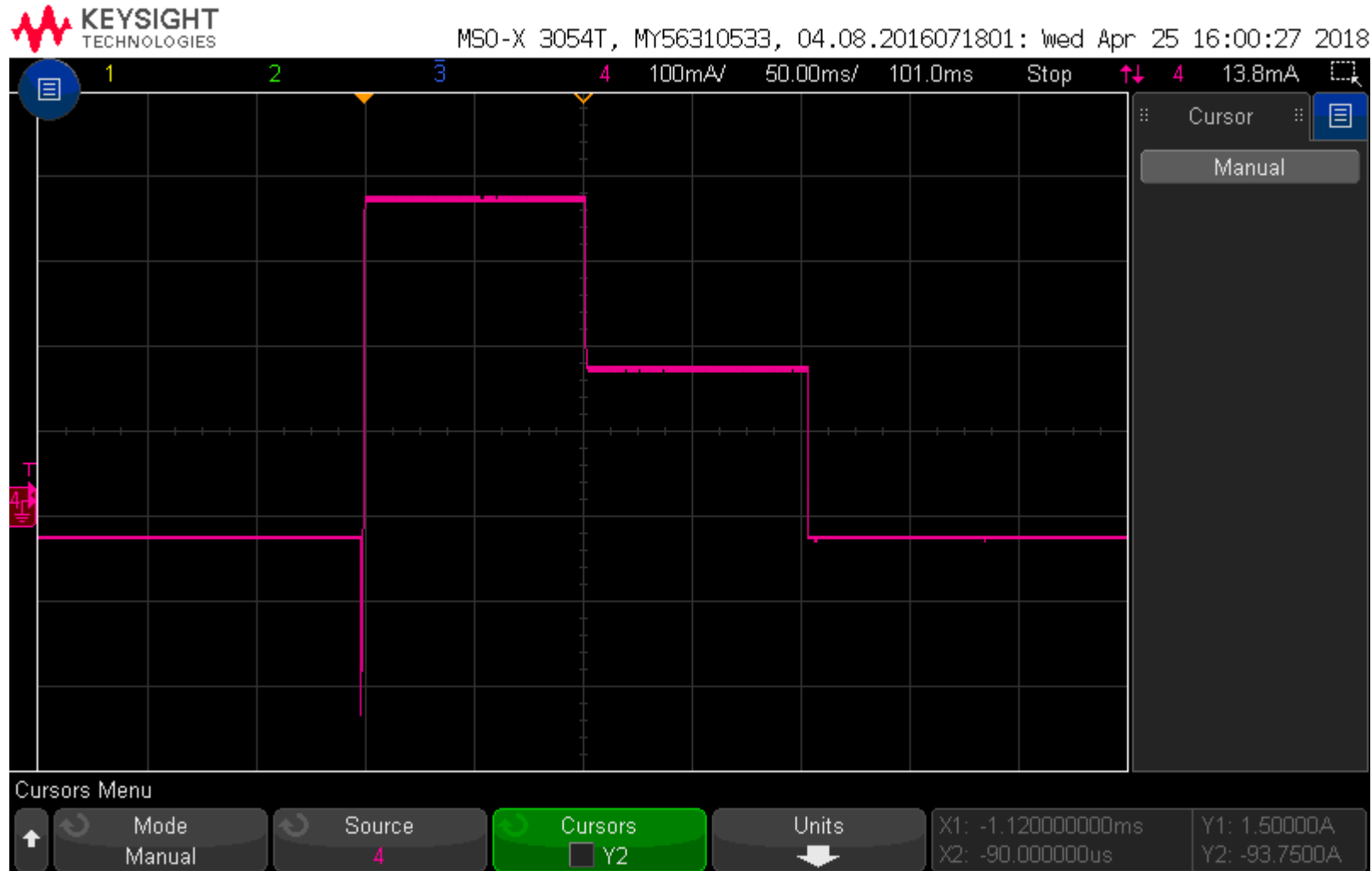
- Set **StartUp** table like this:

Revup Configuration						
	Num	Final Speed (rpm)	Final Torque	Duration (ms)	Last read	Last write
▶	1	0	4000	0	2018-04-25 11:45:57.001	never
	2	0	4000	100	2018-04-25 11:45:57.063	never
	3	0	2000	0	2018-04-25 11:45:57.094	never
	4	0	2000	100	2018-04-25 11:45:57.157	never
	5	0	0	0	2018-04-25 11:45:57.188	never

MC Workbench tuning - Current regulator

41

- The **properly** set current regulator – sharp edges, fast reaction

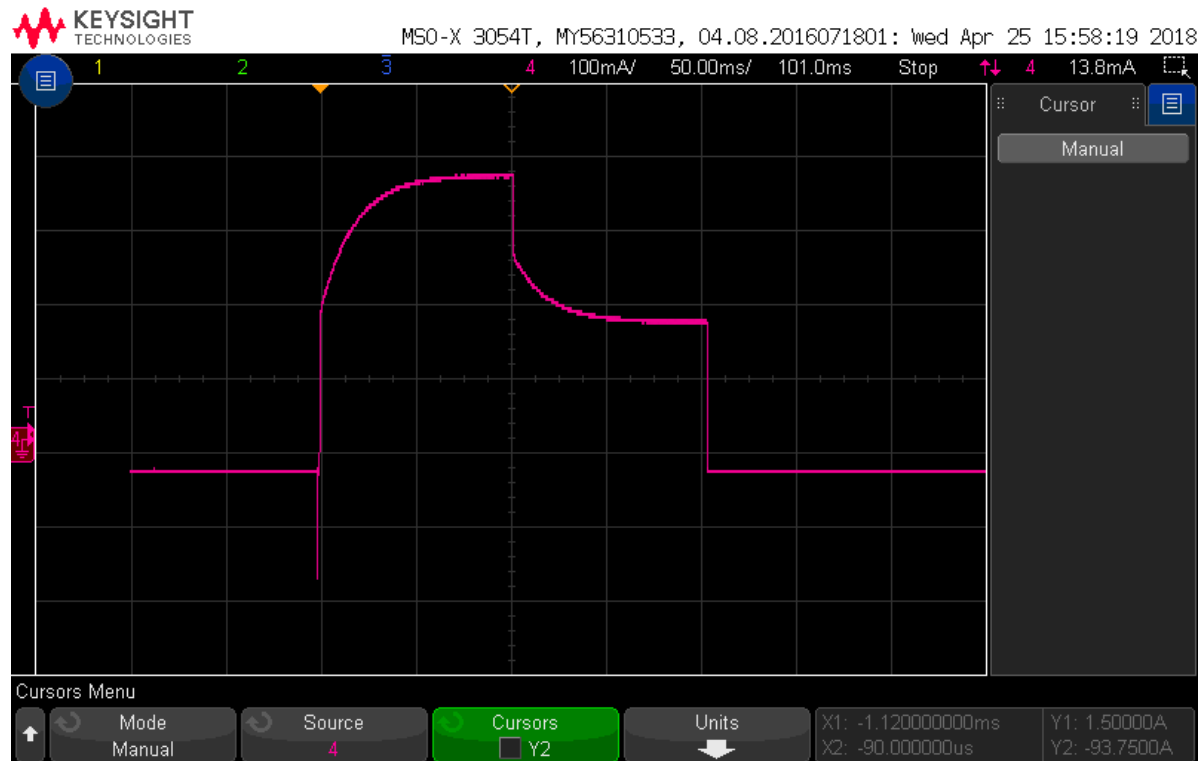


MC Workbench tuning - Current regulator

42

- The **incorrectly** set current regulator

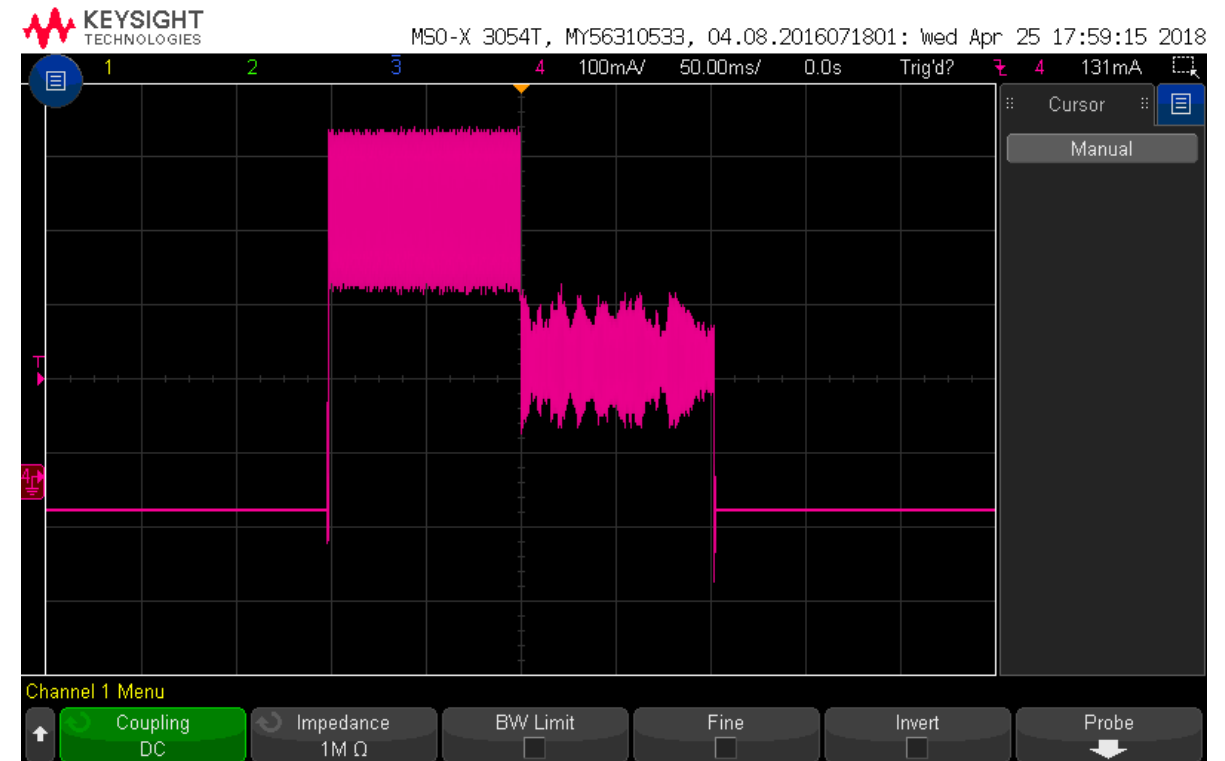
☐ Under-regulated



💡 Increase K_i

 life.augmented

☐ Over-regulated

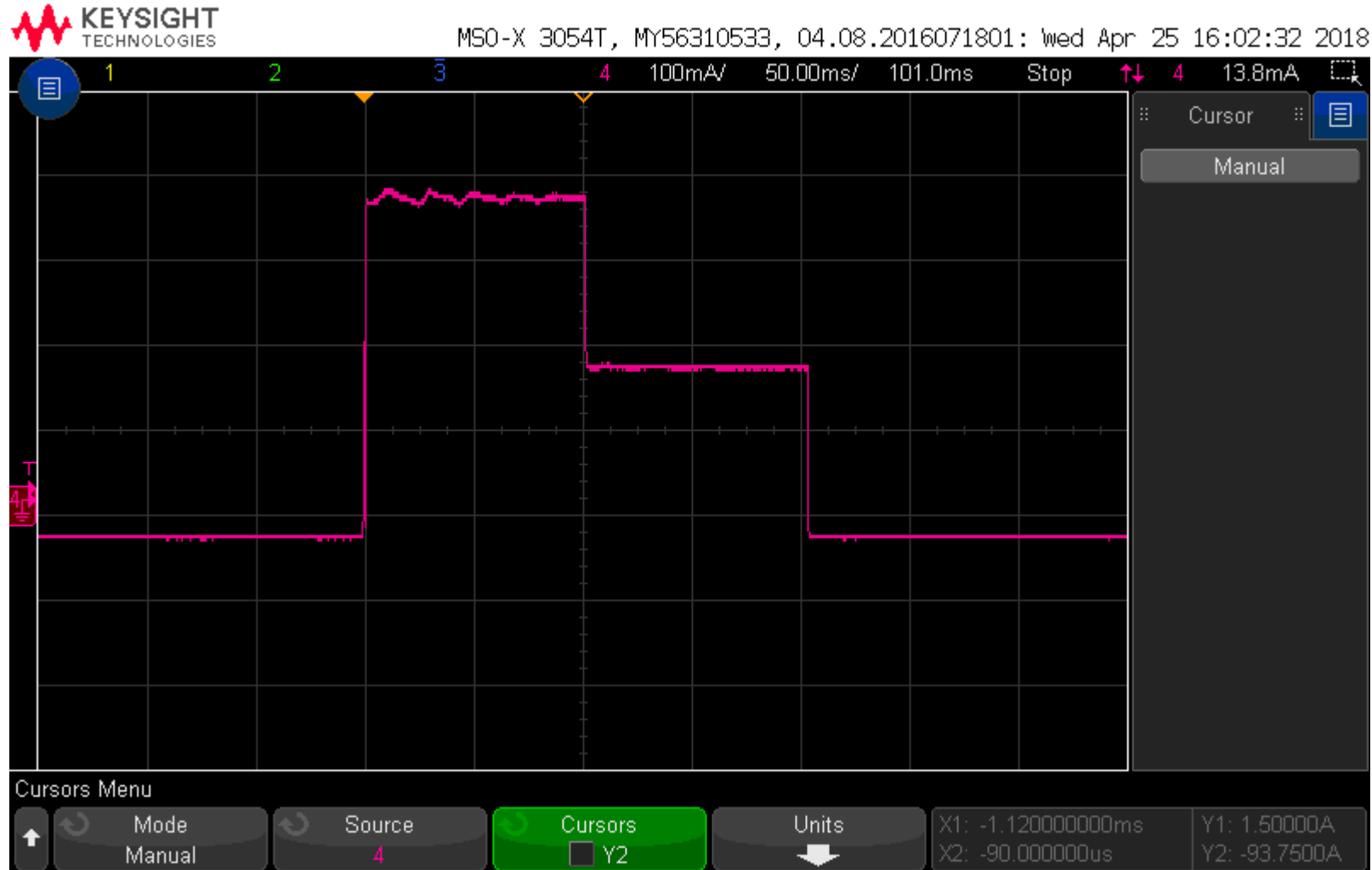


💡 Decrease K_p and increase ratio between K_p/K_i

MC Workbench tuning - Current regulator

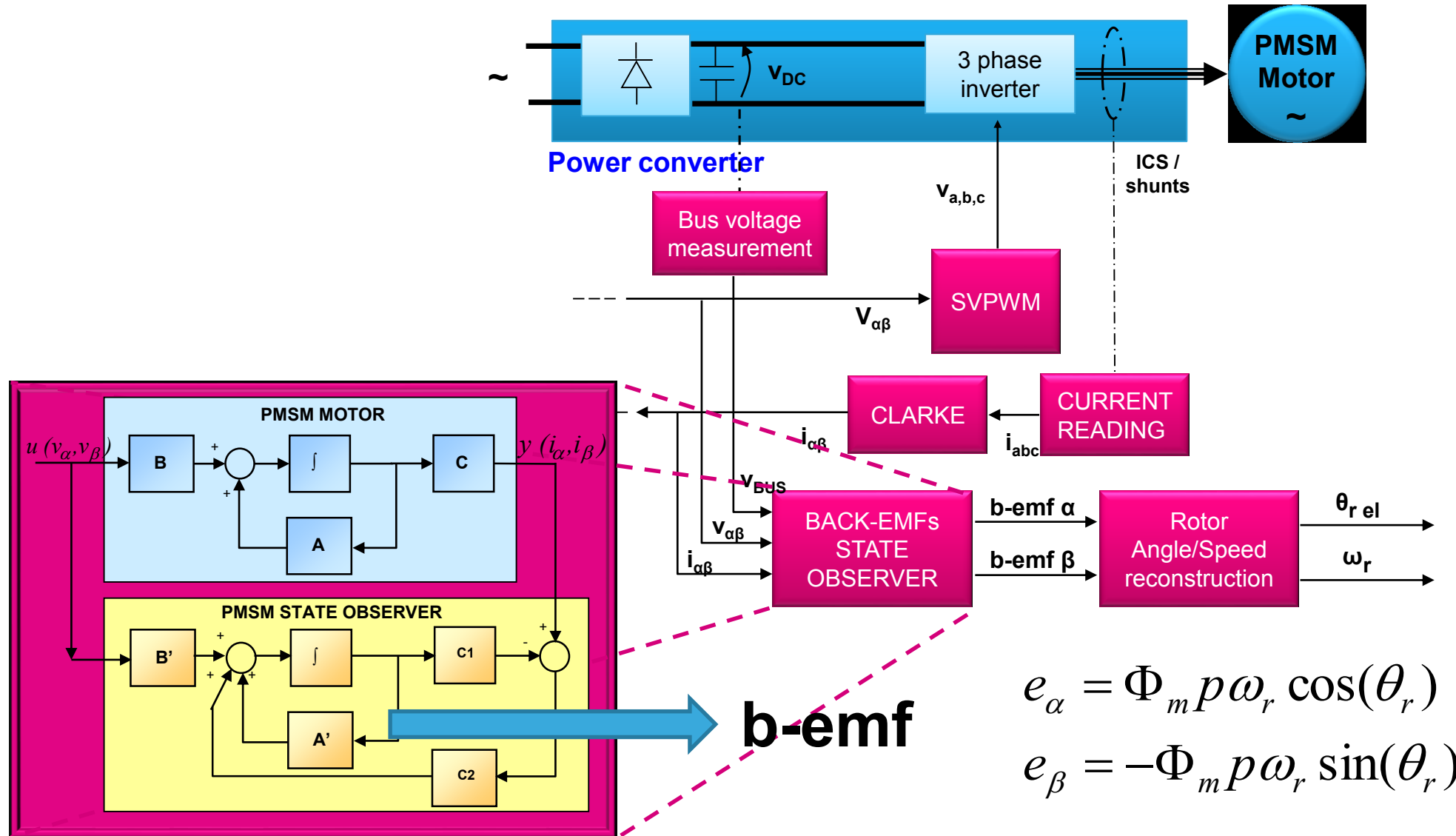
43

- The **properly** set current regulator, but **without fixing** the rotor



MC Workbench tuning - What is Observer?

44



$$e_\alpha = \Phi_m p \omega_r \cos(\theta_r)$$

$$e_\beta = -\Phi_m p \omega_r \sin(\theta_r)$$

MC Workbench tuning - Observer

45

ST Motor Control Workbench [MC_WorkShop_01]

File Tools Help Documentation

Port COM143 115200 Close Monitor

Firmware: ST MC SDK Ver. 5.0.2

Basic Advanced Registers Configuration

Configuration and debug

Control mode Speed

Power Board Status

BUS Voltage 11 Volt

Heatsink temp. 26 °C

DAC Settings

Ch1 Ia

Ch2 Ib

Sensor-less Observer+PLL

Observer C1 -22530

Observer C2 2500

PLL Kp 203

PLL Ki 4

Current controller

Set current reference in speed mode

Torque ref (Iq) 465

Flux ref (Id) 0

Measured currents

Torque (Iq) 69

Flux (Id) 0

Iq PID Gains

Kp 32000

Ki 32000

Id PID Gains

Kp 0

Ki 0

Speed controller

Speed ramp

Target speed 1700 rpm

Duration 0 millsec

Exec ramp

PID Gains

Kp 25500

Ki 200

Sensor-less Observer+Cordic

Observer C1 0

Observer C2 0

Flux weakening tuning

Kp 0

Ki 0

BUS Voltage allowed

Ref 0 %

Meas 0 %

Generic

Target motor

Motor 1

Start Motor

Stop Motor

Stop Ramp

Fault Ack

Encoder Align

All motors

PFC

PFC Enable

PFC Disable

PFC Fault Ack

Monitor

Measured speed (rpm) 0

Variable Motor Uni

PWM frequency 30000 Hz

Sensor selection main Sensor-less (O...

Sensor selection aux Sensor-less (O...

Torque&Flux - Execution rate 1 PW

Bus voltage sensing true

Time Motor Id Message

12:12:57 Command "Start Motor" done!

12:12:59 Command "Fault Ack" done!

12:13:00 Command "Start Motor" done!

01:12:34 Command "Fault Ack" done!

Info / Errors / Warnings Change Log

MC Workbench tuning - Observer

46

- How can you check whether the observer is set properly?
- What do we need?
 - DAC output
 - Oscilloscope
- The **DAC output CH1/CH2** must be set to **Observer BEMF alpha/beta (PLL)**

DAC Settings

Ch1	Observer BEMF alpha (PLL)
Ch2	Ib

MC Workbench tuning - Observer

47

- Observer gain C2 is too high and this makes speed reconstruction a bit noisy (never recognized as reliable). A mix of following solutions could be required:
 1. Decrease original observer gain C2 by successive steps: /2, /4, /6, /8
 2. Enlarge Drive parameters, Speed/position feedback management, variance threshold so as to make rotor locked check less 'demanding' (up to 80% for PLL and 400% for CORDIC)

Sensor-less Observer+PLL	
Observer C1	-22530
Observer C2	2500
PLL Kp	203
PLL Ki	4

MC Workbench tuning - Observer

48

- Default parameters, taken from Motor Profiler measuring -> noisy BEMF ☹️

- Measuring current (probe)
- DAC output – observed B-EMF

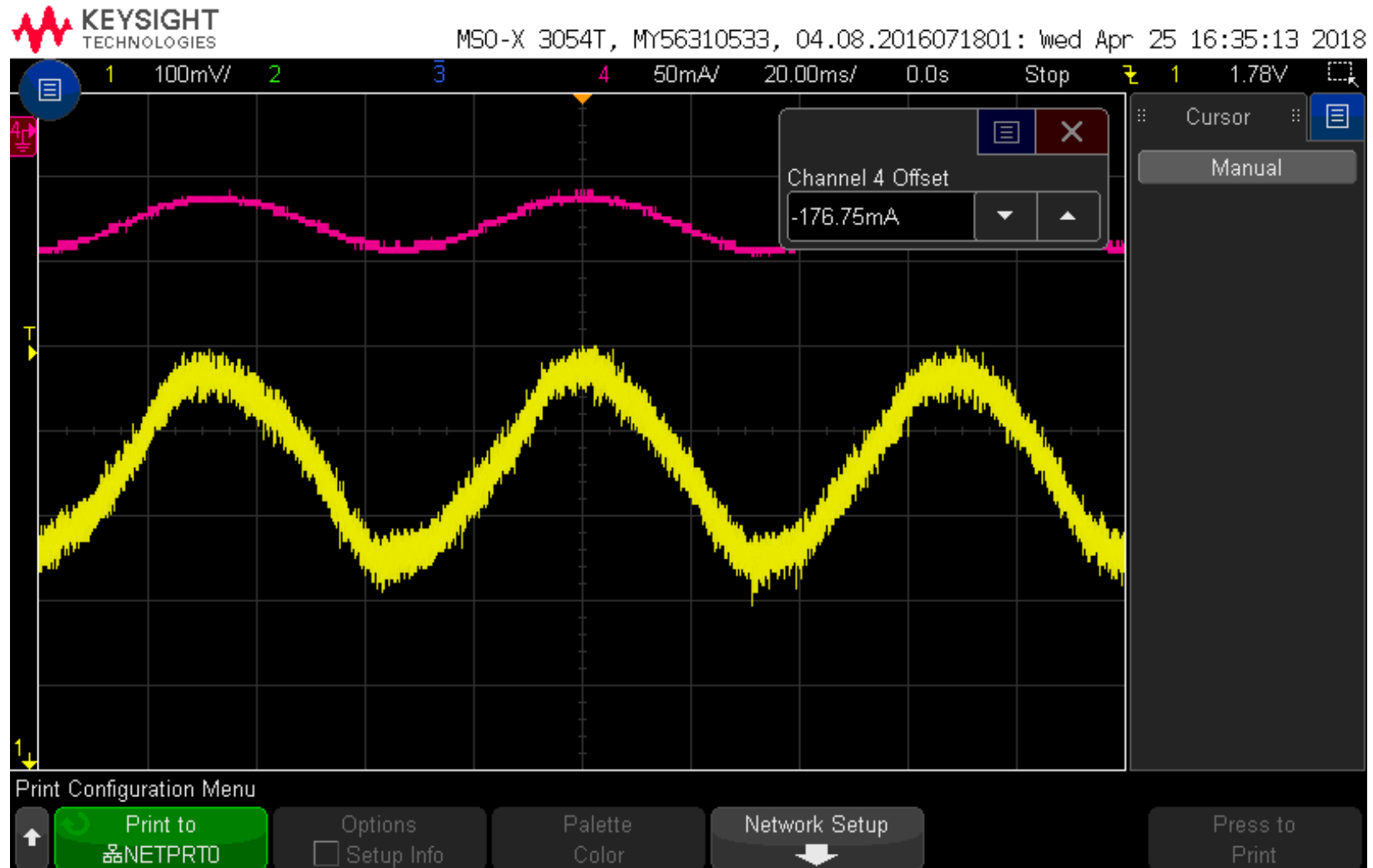
Sensor-less Observer+PLL

Observer C1 -22530

Observer C2 18682

PLL K_p 203

PLL K_i 4



MC Workbench tuning - Observer

49

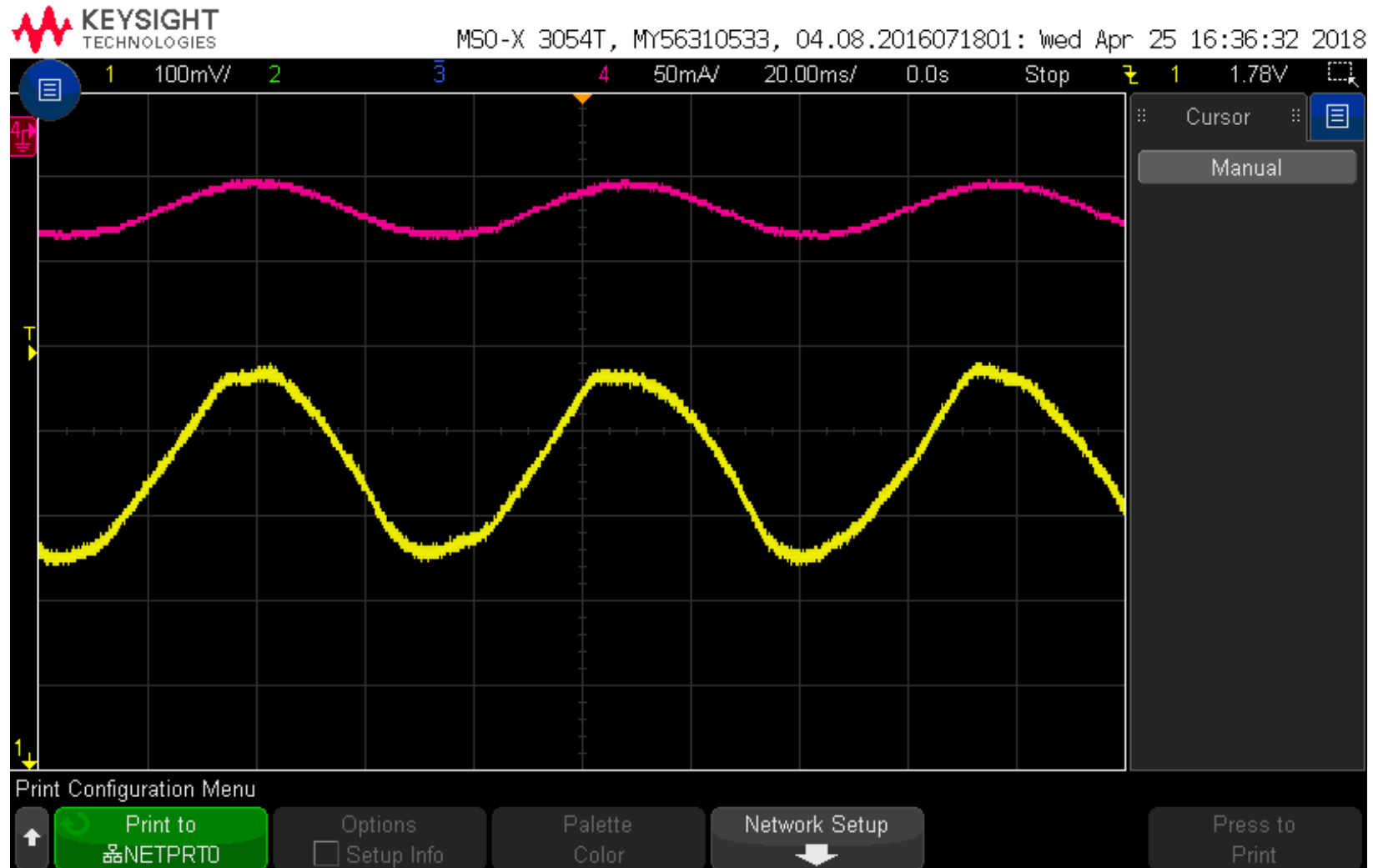
- Reduced C2 parameter -> improved BEMF (filtered out) 😊

- Measuring current (probe)
- DAC output – observed B-EMF

Sensor-less Observer+PLL

Observer C1	-22530
Observer C2	2500
PLL K _p	203
PLL K _i	4

💡 Decrease C2



MC Workbench tuning - Observer

50

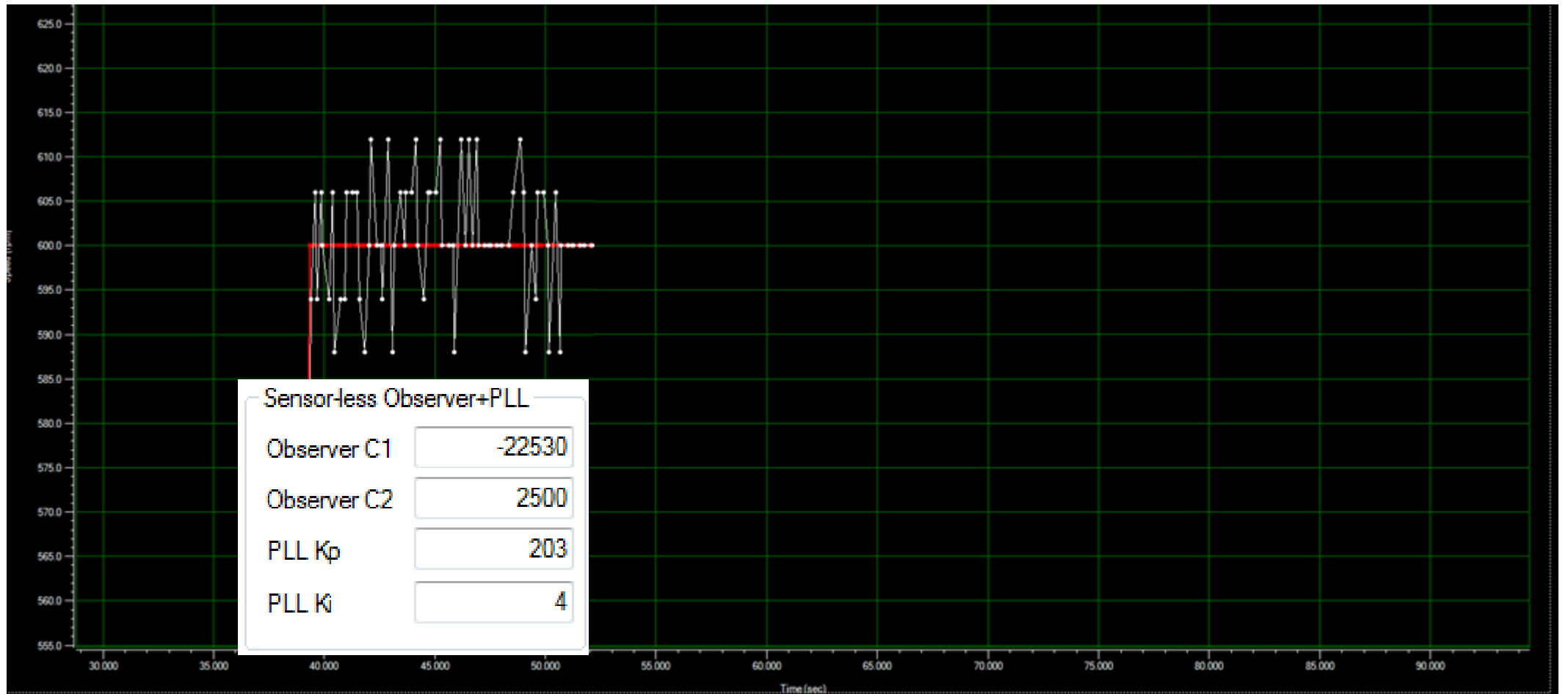
- PLL Kp and Ki are sometimes too high, you can recognize this situation during low speed operation on measured speed chart
 - The solution is same like for C2, decrease both parameters by successive steps: /2, /4, /6, /8.

Sensorless Observer+PLL	
Observer C1	-22530
Observer C2	2500
PLL Kp	203
PLL Ki	4

MC Workbench tuning - Observer

51

- Record from MC Workbench Monitor → noisy speed reconstruction 😞



MC Workbench tuning - Observer

52

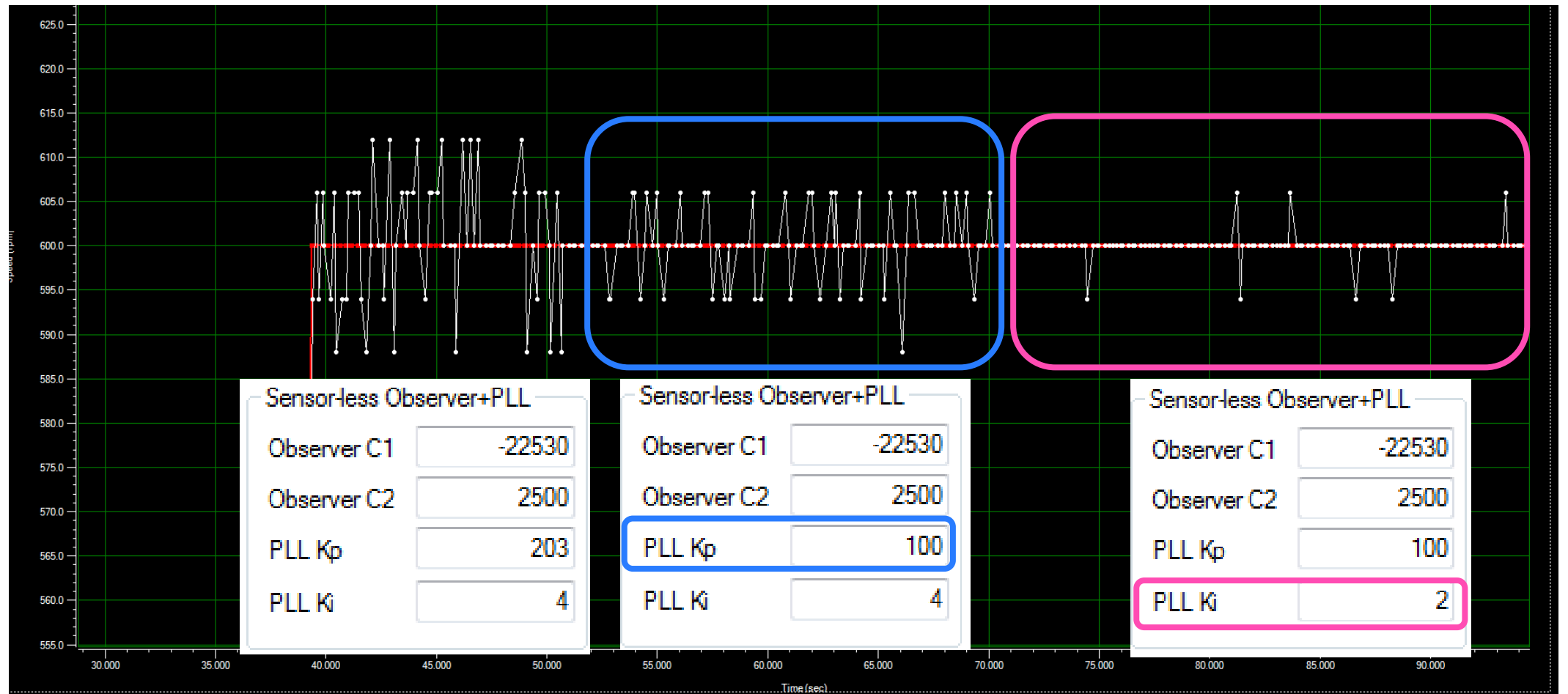
- Record from MC Workbench Monitor → slightly improved speed reconstruction ☺



MC Workbench tuning - Observer

53

- Record from MC Workbench Monitor → well improved speed reconstruction 😊



MC Workbench tuning - Speed regulator

54

ST Motor Control Workbench [MC_WorkShop_01]

File Tools Help Documentation

Port COM143 115200 Close Monitor

Firmware: ST MC SDK Ver. 5.0.2

Basic Advanced Registers Configuration

Configuration and debug

Control mode Speed

Power Board Status

BUS Voltage 11 Volt

Heatsink temp. 26 °C

DAC Settings

Ch1 Ia

Ch2 Ib

Current controller

Set current reference in speed mode

Torque ref (Iq) 465

Flux ref (Id) 0

Measured currents

Torque (Iq) 69

Flux (Id) 0

Iq PID Gains

Kp 32000

Ki 32000

Id PID Gains

Kp 0

Ki 0

Speed controller

Speed ramp

Target speed 1700 rpm

Duration 0 ms

Exec ramp

PID Gains

Kp 25500

Ki 200

Sensor-less Observer+PLL

Observer C1 -22530

Observer C2 2500

PLL Kp 203

PLL Ki 4

Sensor-less Observer+Cordic

Observer C1 0

Observer C2 0

Flux weakening tuning

Kp 0

Ki 0

BUS Voltage allowed

Ref 0 %

Meas 0 %

Generic

Target motor

Motor 1

Start Motor

Stop Motor

Stop Ramp

Fault Ack

Encoder Align

All motors

PFC

PFC Enable

PFC Disable

PFC Fault Ack

Monitor

Measured speed (rpm) 0

Variable Motor Uni

PWM frequency 30000 Hz

Sensor selection main Sensor-less (O...

Sensor selection aux Sensor-less (O...

Torque&Flux - Execution rate 1 PW

Bus voltage sensing true

Time Motor Id Message

12:12:57 Command "Start Motor" done!

12:12:59 Command "Fault Ack" done!

12:13:00 Command "Start Motor" done!

01:12:34 Command "Fault Ack" done!

Info / Errors / Warnings Change Log

MC Workbench tuning - Speed regulator

55

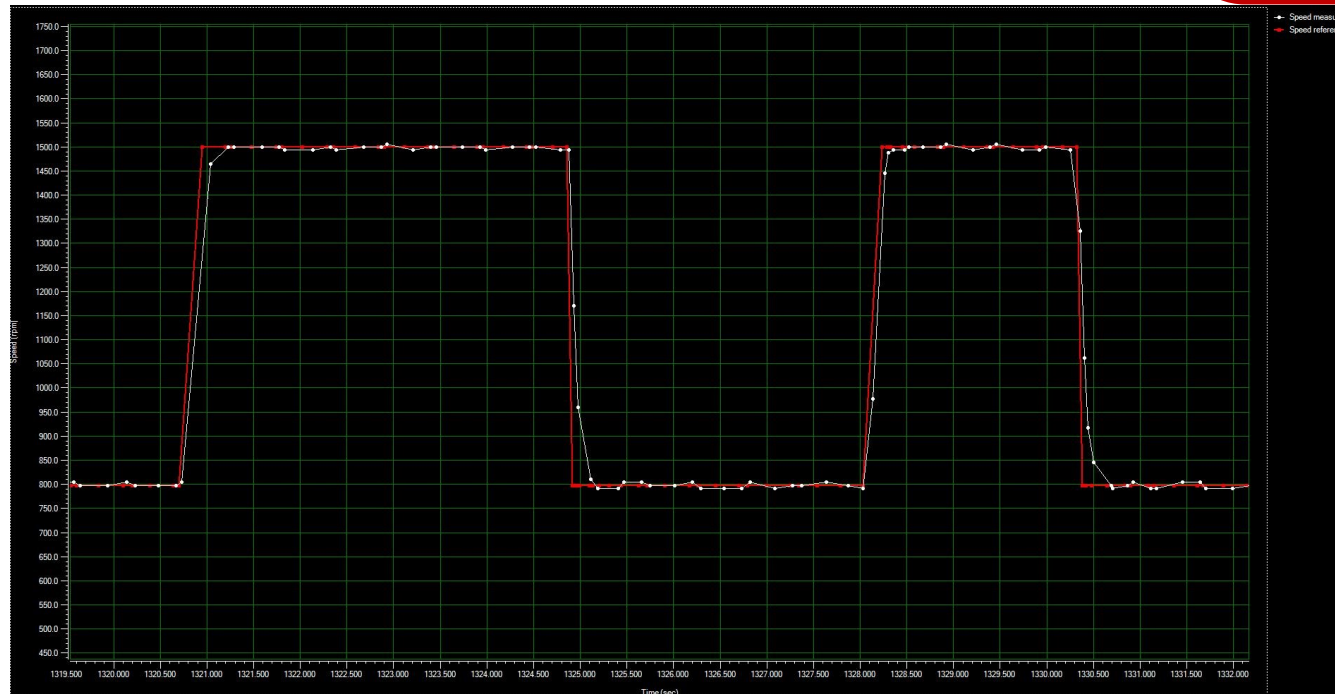
- How can you check whether the speed regulator is set properly?
- Check it directly in the MC Workbench Monitor
- Set speed ramp with zero duration and target speed
- Observe a response curve in the Plotter

Speed ramp

Target speed rpm

Duration millisecc

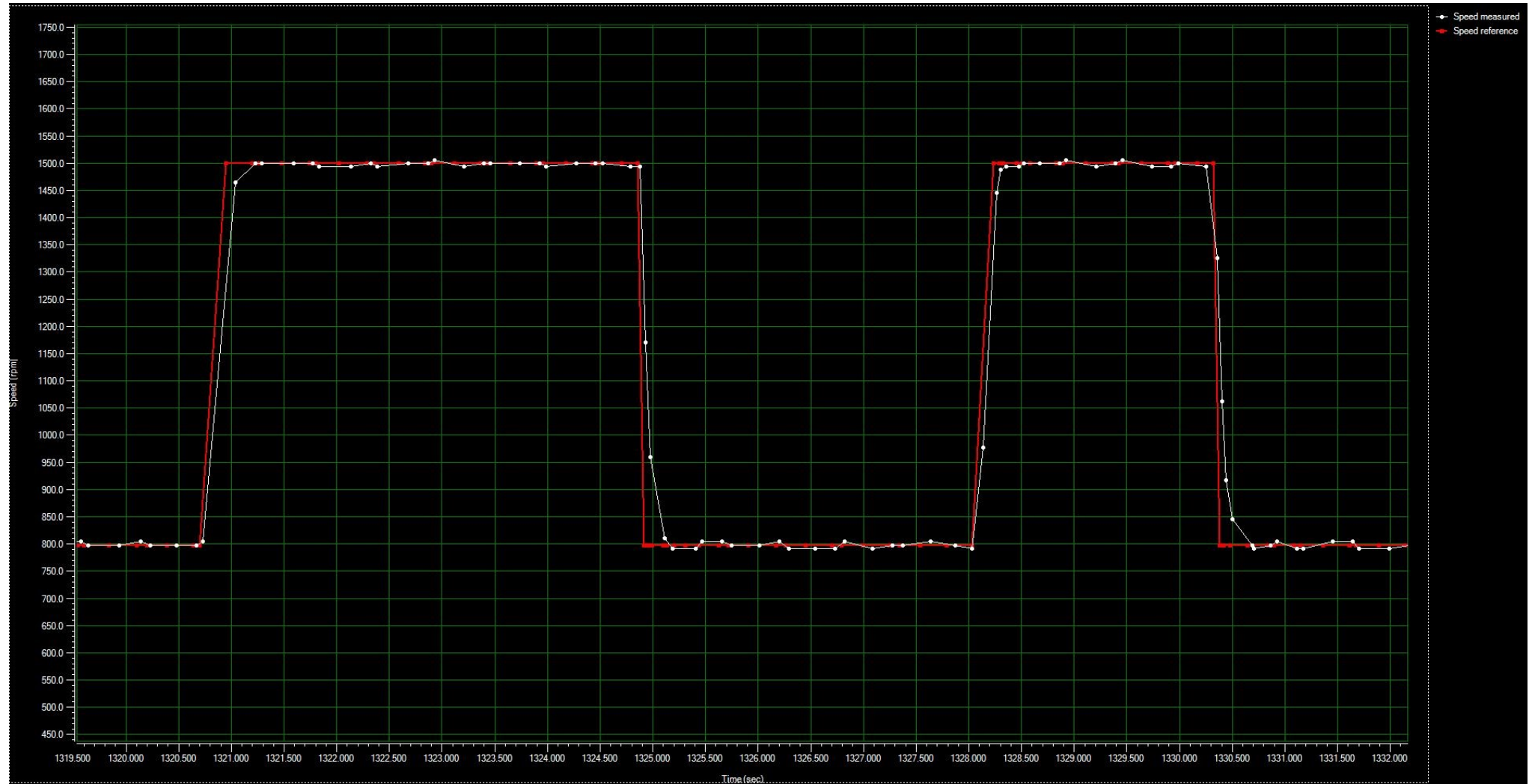




MC Workbench tuning - Speed regulator

56

- The **properly** set speed regulator



MC Workbench tuning - Speed regulator

57

- The **incorrectly** set speed regulator – over-regulated



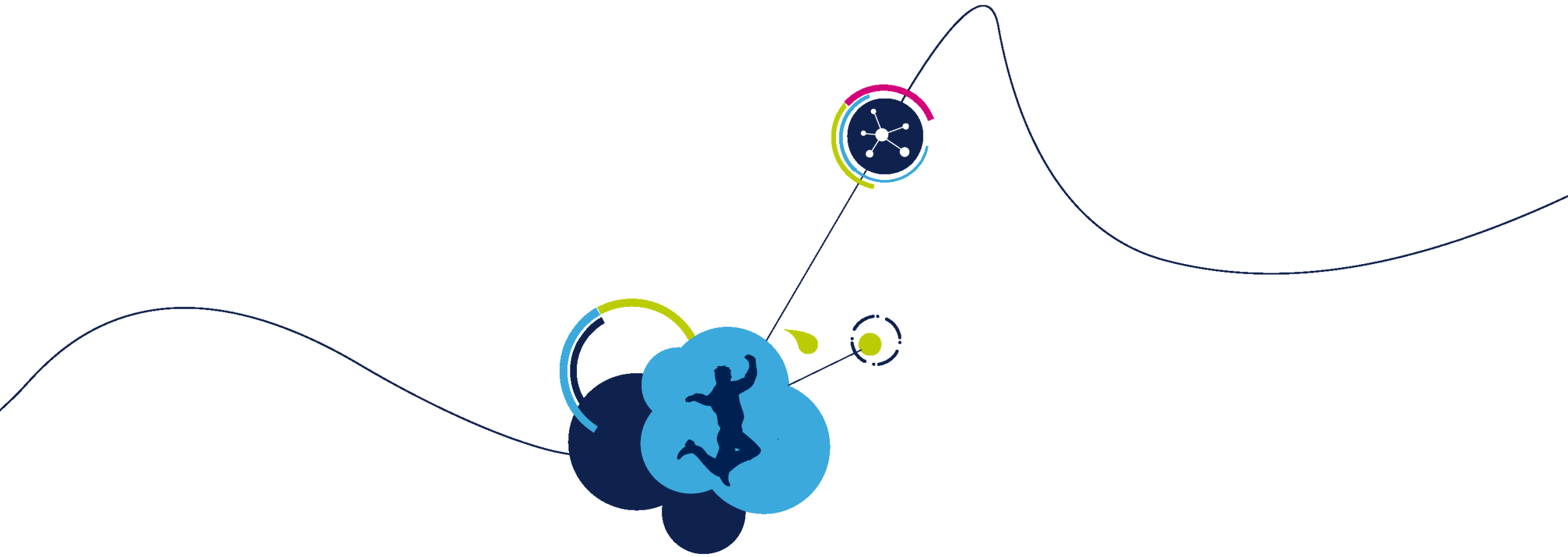
MC Workbench tuning - Speed regulator

58

- The **incorrectly** set speed regulator – under-regulated



Increase Ki



General tips for tuning PI regulators

General tips for tuning PI regulators

60

- There exist many methods for proper tuning, let's start with manual
- The tuning requires a step change in the required quantity (speed, load, current) and measurement of the regulated quantity response
- An example is change of the required speed as a step from 2000 RPM to 4000 RPM. The response is the real speed over time:



General tips for tuning PI regulators

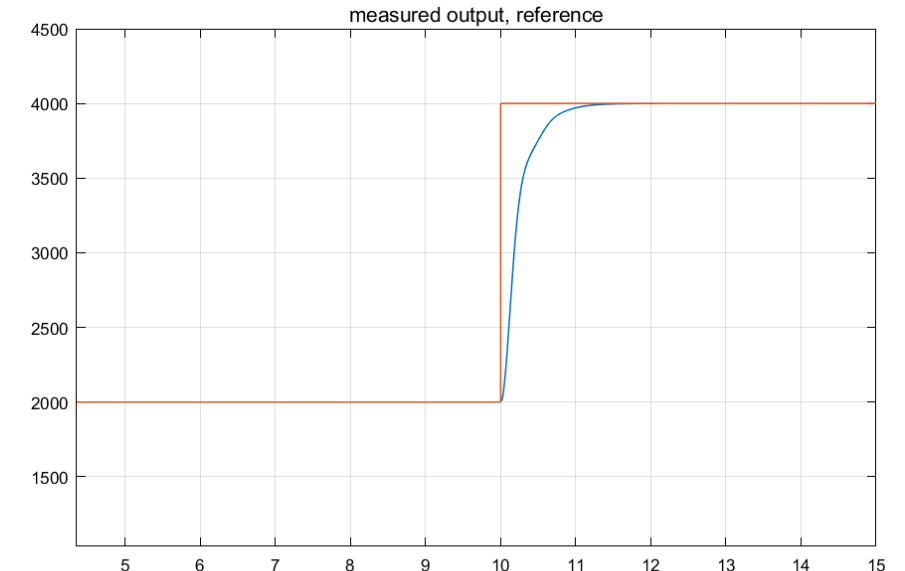
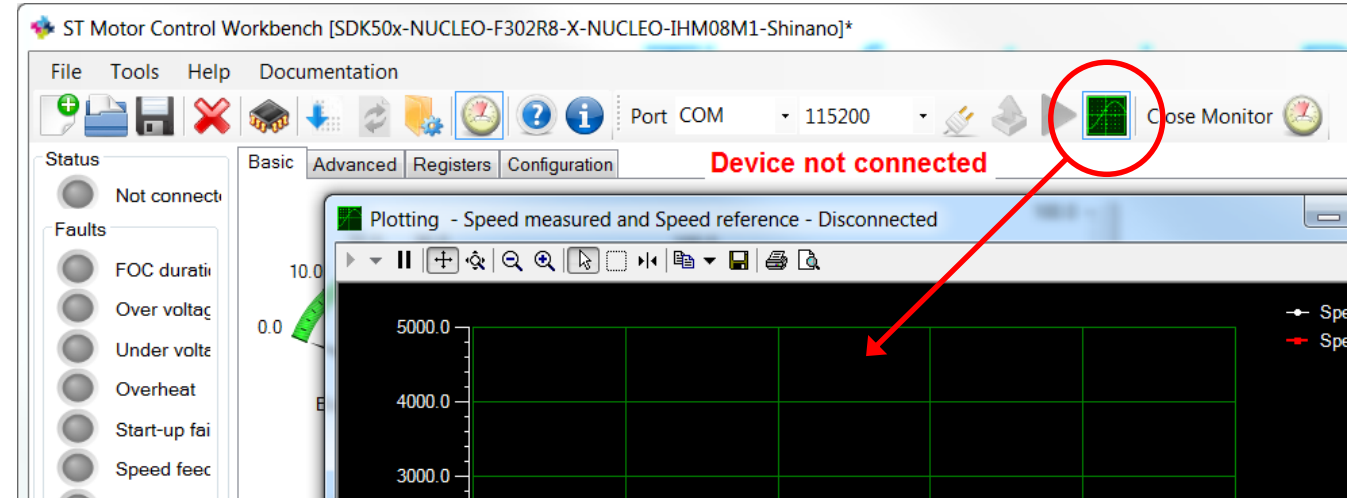
61

- On previous slide, we saw damped oscillation response – this is usually unwanted behavior, so let's mention some rules of thumb!
- If there is an overshoot, change the ratio K_p/K_i to avoid it or vice versa
- If the shape is exponentially closing to the required value, keep the ratio K_p/K_i , but proportionally change both K_p and K_i to increase or decrease the slope.
- Try this at different speeds and loads and choose conservative estimation – avoid undamped oscillation at all costs!

General tips for tuning PI regulators

62

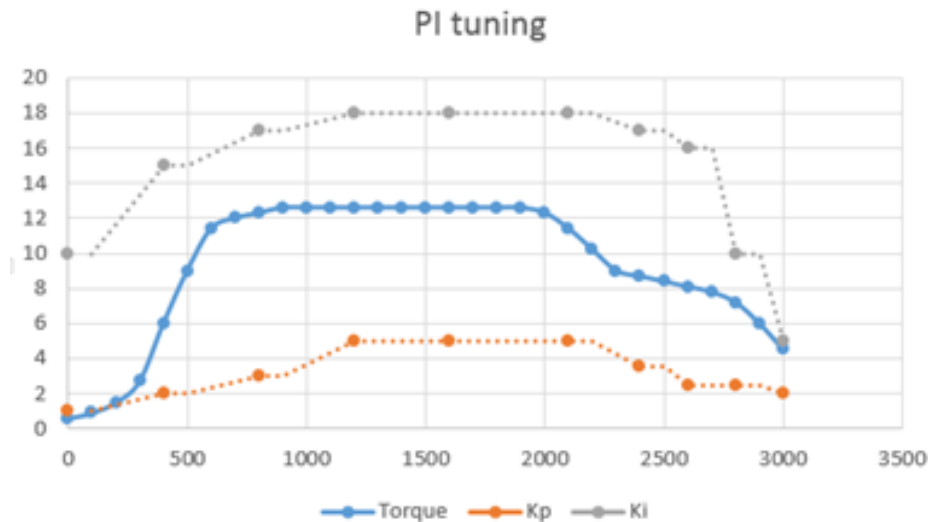
- Such chart can be displayed in MC workbench, Monitor mode, via Plotter window
- Otherwise use DACs and oscilloscope
- Optimum reaction, if overshoot is undesired, is on the following picture:



General tips for tuning PI regulators

63

- If the load torque changes with speed, different tuning may be needed at different speeds. An example of optimal value of K_p , K_i vs. speed is on the following picture:



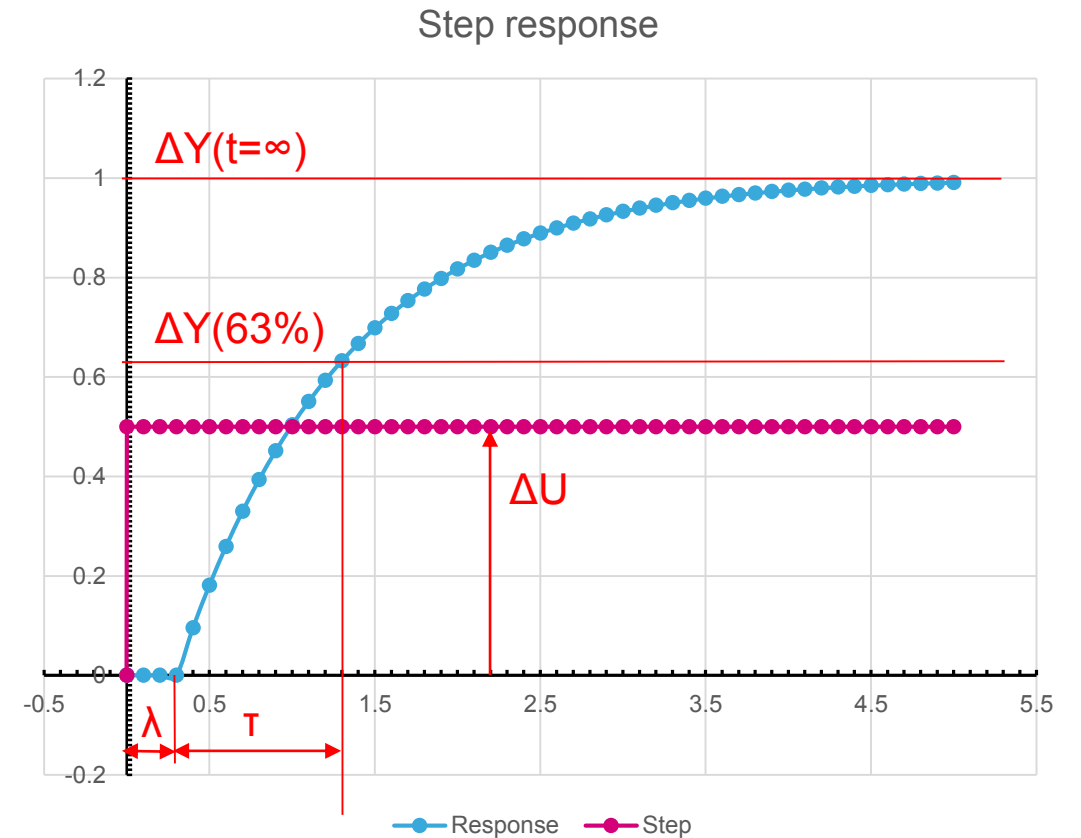
- Such K_p , K_i interpolation may need to be implemented in user SW for the Speed PI

General tips for tuning PI regulators

64

- Another more analytical methods need some measurements in the torque mode:

- Determine stabilized speed response $Y(t=\infty)$ on torque step U :
- Determine Process dead time (λ)
- Determine Time constant (τ) when speed crosses 63% of the speed step (between stabilized value and initial speed)
- Read PI process frequency f_s (usually 2 kHz)



General tips for tuning PI regulators

65

- Typically the motor with connected load is a first order system (with simple exponential response) with process delay λ
- In such case, simple calculation is needed:
 - Process gain $K = \frac{\Delta Y(t=\infty)}{\Delta U}$
 - Here the ΔY has unit of *dpp* (digits per PWM) and ΔU is change in Compare register of MC timer
- Let's consider PI(D) regulator of this form:

$$y(s) = K_p \left(1 + \frac{1}{T_i \cdot s} + T_d \cdot s \right) e(s)$$

General tips for tuning PI regulators

66

- Ziegler-Nichols method (overshoot, aggressive)

Type	K_p	τ_i	τ_d
P	$\frac{1}{K} \frac{\tau}{\lambda}$	∞	0
PI	$\frac{0.9}{K} \frac{\tau}{\lambda}$	$3.33 \cdot \lambda$	0
PID	$\frac{1.2}{K} \frac{\tau}{\lambda}$	$2.0 \cdot \lambda$	$0.5 \cdot \lambda$

General tips for tuning PI regulators

67

- Cohen-Coon method (moderate to conservative)

Type	K_p	T_i	T_d
P	$\frac{1}{K} \frac{\tau}{\lambda} \left(1 + \frac{1}{3} \frac{\lambda}{\tau} \right)$	∞	0
PI	$\frac{1}{K} \frac{\tau}{\lambda} \left(0.9 + \frac{1}{12} \frac{\lambda}{\tau} \right)$	$\lambda \left(\frac{30 + 3 \frac{\lambda}{\tau}}{9 + 20 \frac{\lambda}{\tau}} \right)$	0
PID	$\frac{1}{K} \frac{\tau}{\lambda} \left(\frac{4}{3} + \frac{1}{4} \frac{\lambda}{\tau} \right)$	$\lambda \left(\frac{32 + 6 \frac{\lambda}{\tau}}{13 + 8 \frac{\lambda}{\tau}} \right)$	$\lambda \left(\frac{4}{11 + 2 \frac{\lambda}{\tau}} \right)$

General tips for tuning PI regulators

68

- Conversion to discrete PI(D) regulator with sampling frequency f_s

$$K'_p = K_p$$

$$K'_i = \frac{K_p}{\tau_i} \frac{1}{f_s}$$

$$K'_d = K_p \tau_d f_s$$

- Calculated values shall be expressed as ratios $\frac{K'_x}{2^N}$ and put in MC workbench, K'_x being in 16-bit range (± 32767)

General tips for tuning PI regulators

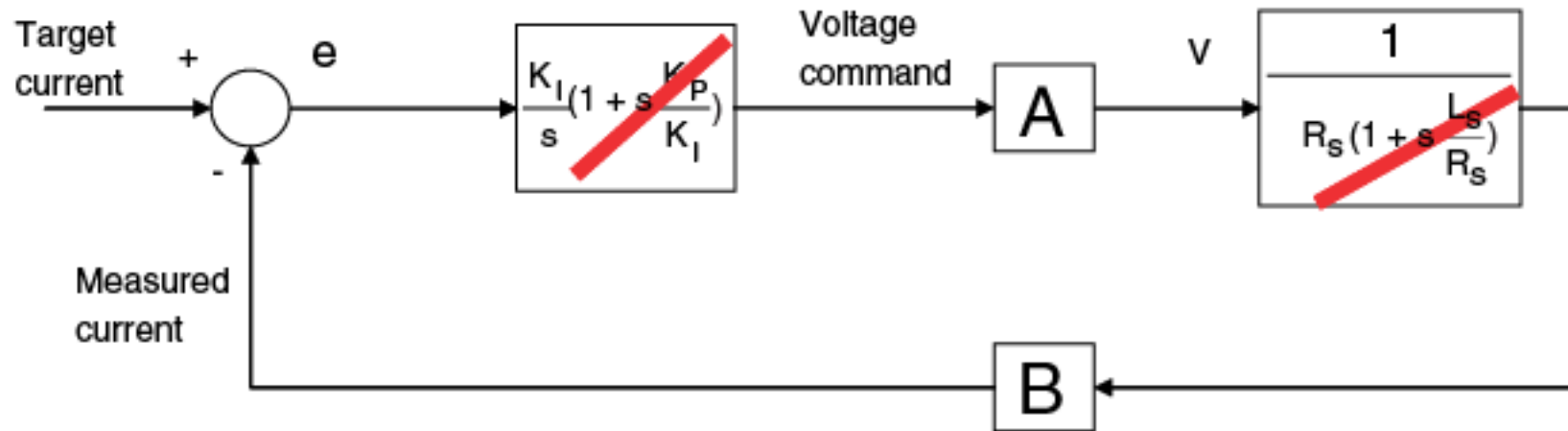
69

- After calculating the PI coefficients, one still needs to tune them manually
- These equations are valid in the range $0.1 < \frac{\lambda}{\tau} < 1$, otherwise other type of tuning is needed

General tips for tuning PI regulators

70

- Another method works in time domain with a priori knowledge of motor and load inertia ($\sim L_s$) a mechanical resistance ($\sim R_s$)
- By substituting K_p/K_i with L_s/R_s ratio, one can perform pole-zero cancellation as shown below, calculating K_p as on previous slide:



ai14852

General tips for tuning PI regulators

71

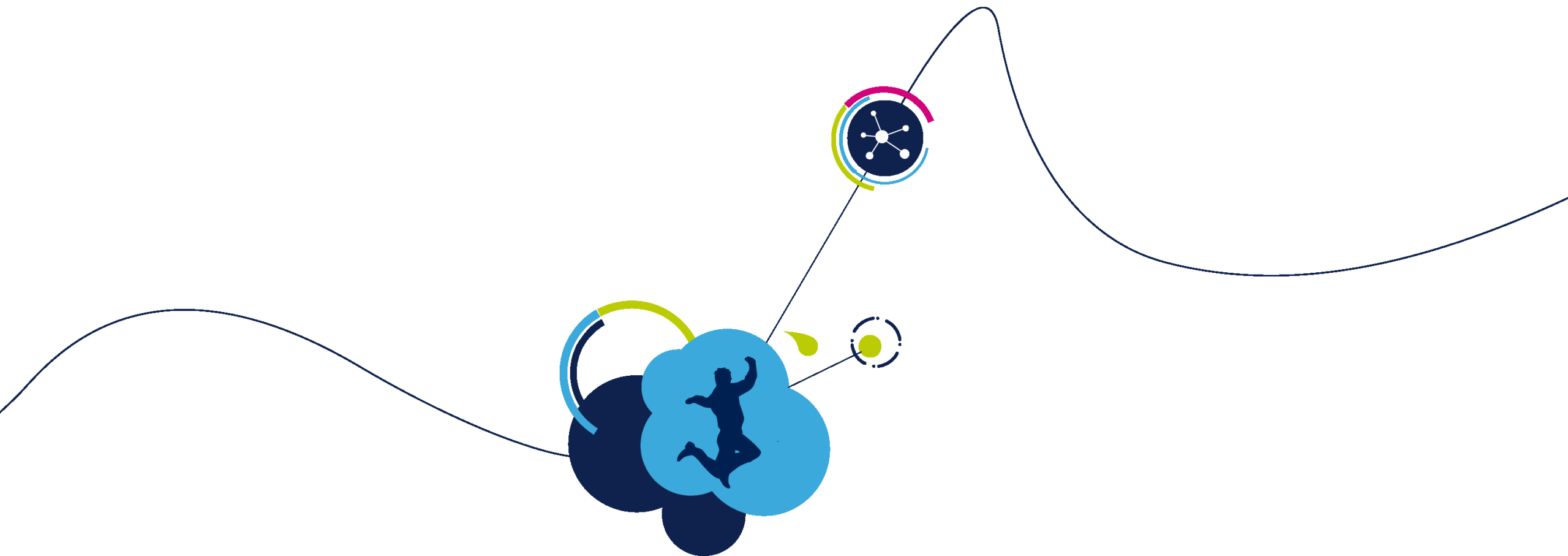
- In case of second-order and more complex or interlinked systems (combined exponential behavior), the response measurement and calculation are complex and discussed in control theory literature, i.e. zero-pole mapping regulators
- Such example shall use PID regulator with derivative component or even more complex (polynomial) regulators
- Small filter with $\tau' = 0.1\tau$ for the error component may be introduced before PI(D) regulator, too

General tips for tuning PI regulators

72

Used literature:

- <http://educyclopedia.karadimov.info/library/pidtune2.pdf>
- http://www.kirp.chtf.stuba.sk/moodle/pluginfile.php/66882/mod_resource/content/0/tidsdiskret_pid_reg.pdf
- <https://pdfs.semanticscholar.org/116c/e07bcb202562606884c853fd1d19169a0b16.pdf>
- Matlab: pidtune, pidtool
- <https://www.biricha.com>

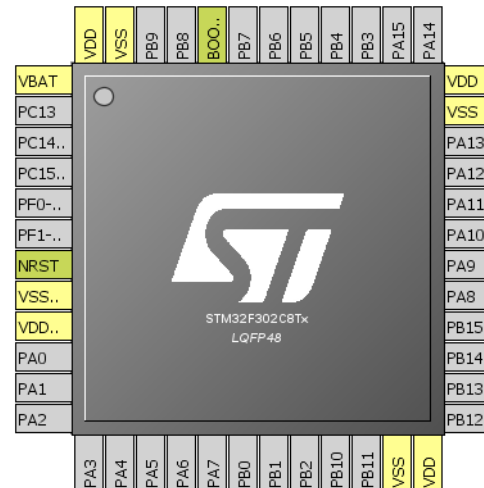
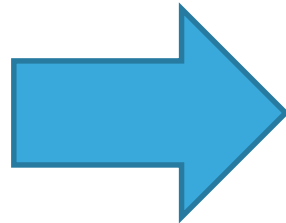
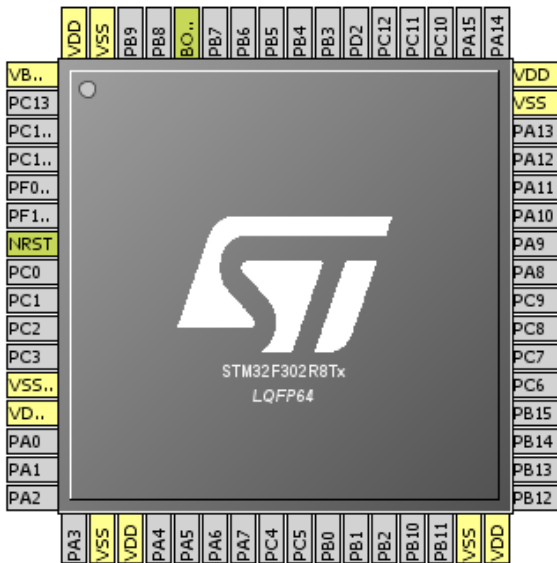


Developing your own HW with
ST MC SDK

Development using own board with different STM32

74

- Create new project with Custom board option
- As example we will move from 64pins to 48 pins



New Project

1 Application type: Custom

2 System: ☒ Single Motor ☐ Dual

3 Select Boards: ☐ Inverter ☐ MC Kit ☒ Power & Control

Control: custom board (Control board where the control stage parameters have to be customized by the user)

Power: custom board (Power board where the power stage parameters have to be customized by the user)

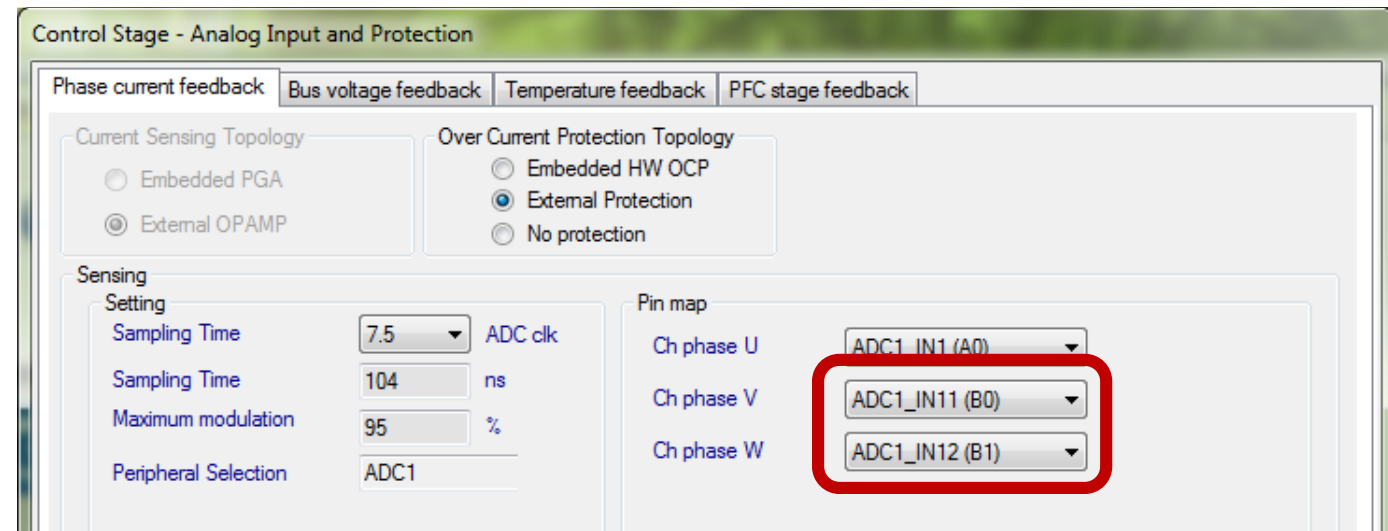
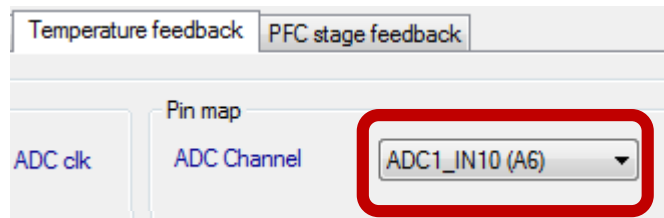
4 Motor: GimBal (IPowerMotor GBM2804-100T)

Magnetic structure: Surface Mounted
Pole Pairs: 7
Nominal Speed: 1614 rpm
Nominal Voltage: 11.9 V
Nominal Current: 1.5 Apk

Development using own board with different STM32

75

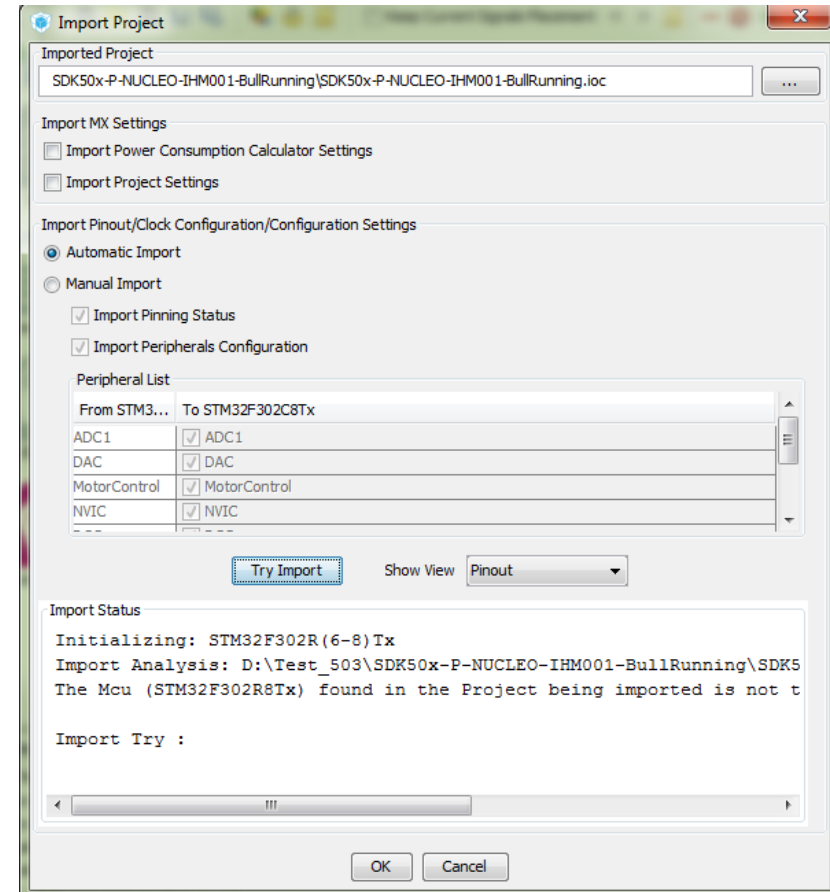
- We have to check Analog and Digital pins in Motor Control Workbench, if they are available in smaller package?
- In our example we are missing analog pins C0, C1, C2 and digital pins C10, C11, C12. We have to select proper replacement PB0, PB1, PA6 for analog and B13,B14,B15 for digital in ST Motor Control Workbench
- Generate the project



Development using own board with different STM32

76

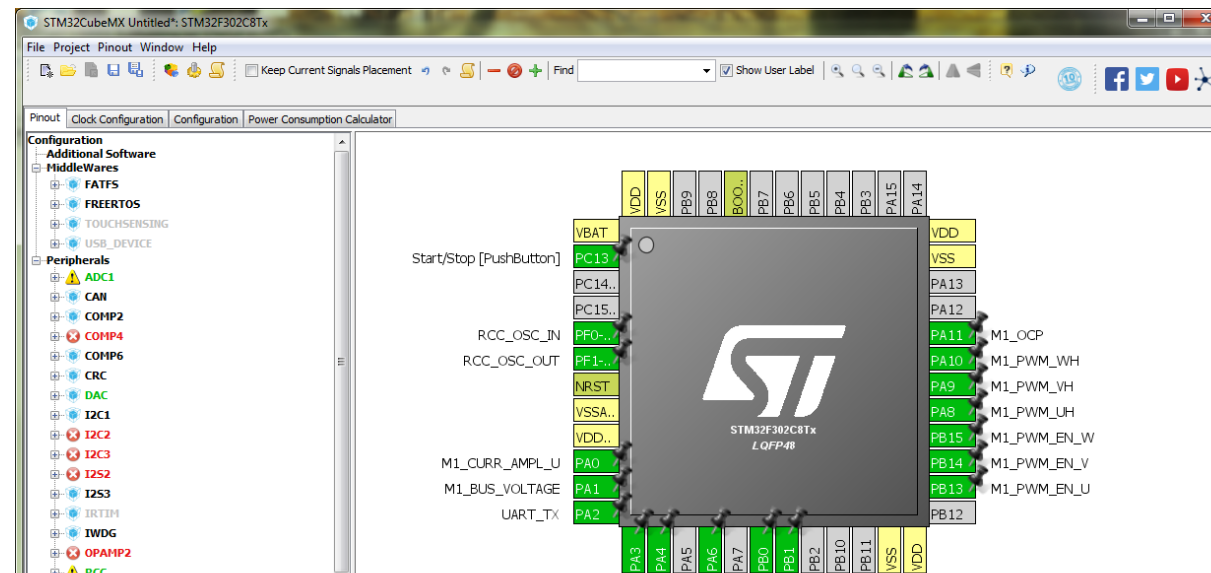
- Create New Project in the STM32CubeMX with new STM32 MCU
- Import project .ioc file which was created by ST Motor Control Workbench
- Click to “**Try Import**”
 - Check the Import Status
 - Click OK



Development using own board with different STM32

77

- Import status is visible in the output window
- In our case we have to manually set RCC tree



Import :

Importing project completed

❌ Import RCC : FAILED; Set STM32F302C8Tx clock tree as 'DEFAULT'.

Development using own board with different STM32

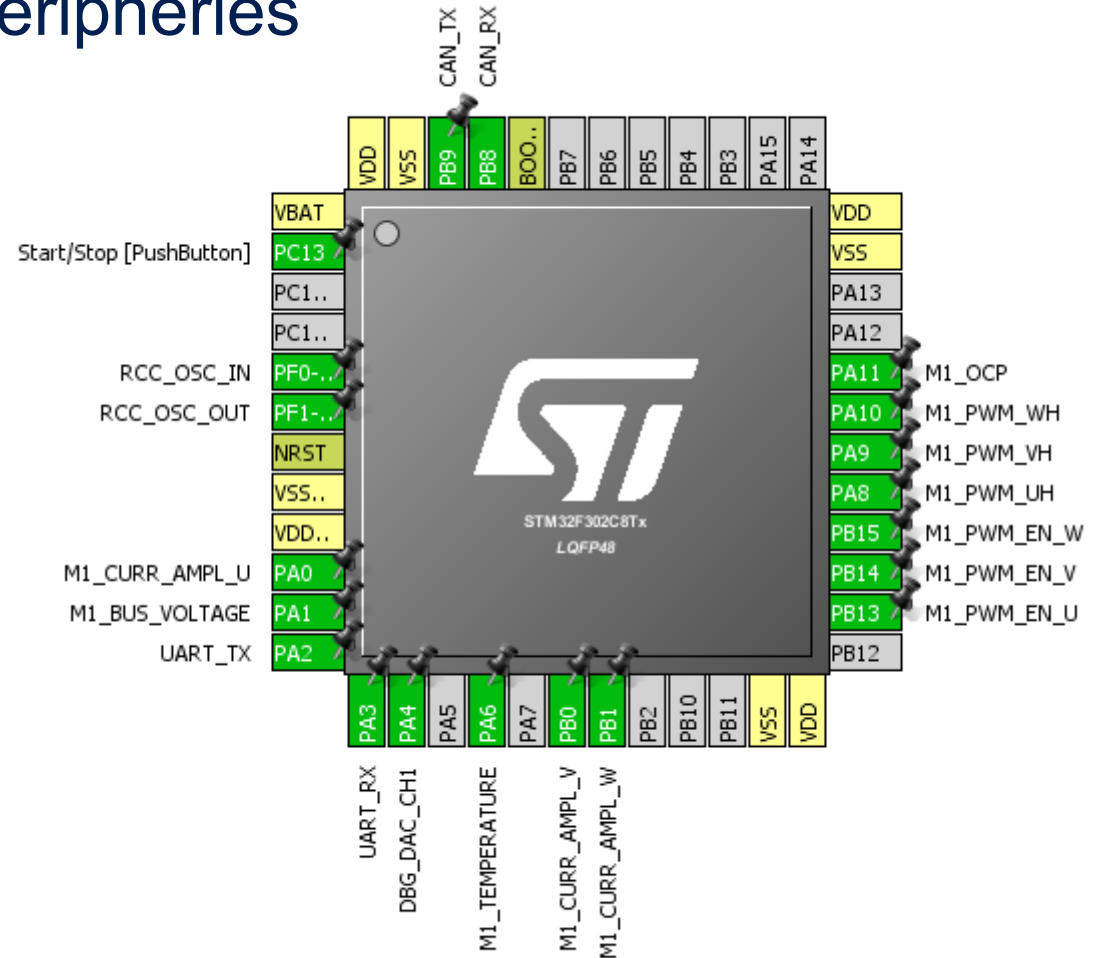
78

- Additionally we can add other peripherals

- USART
- CAN
- ETH
- LCD

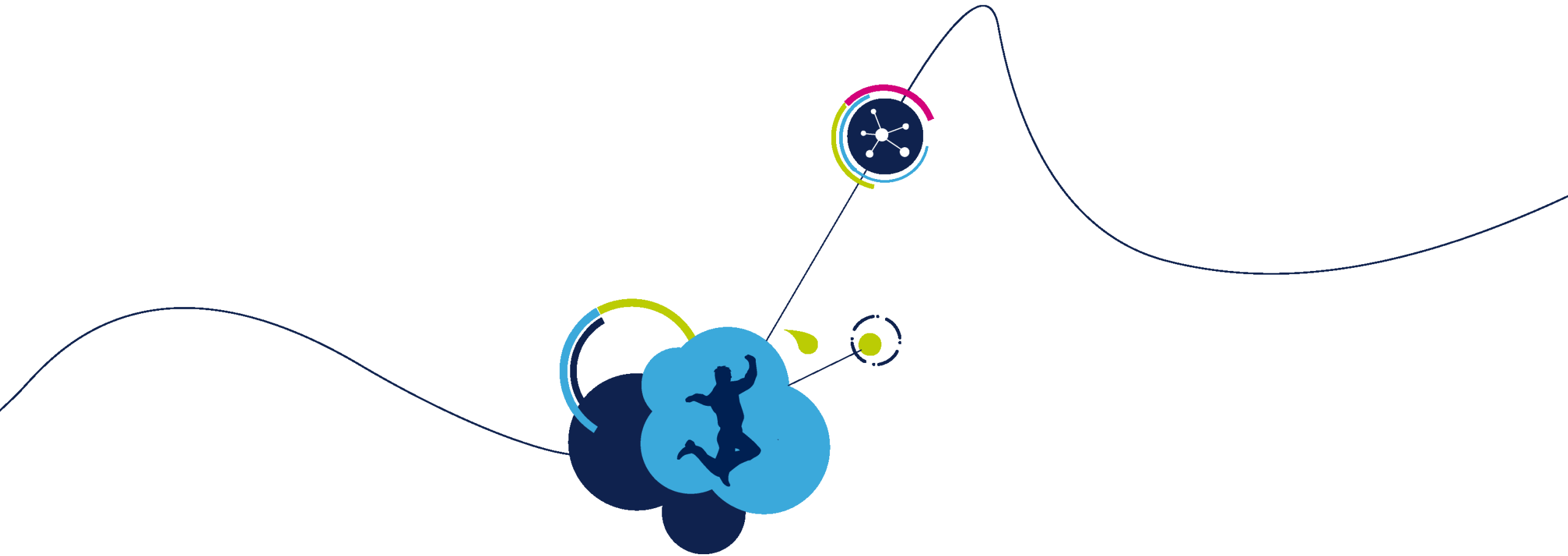
- Save new project

- Generate a code



- You can find additional Tips & Tricks in F.A.Q. located on ST website!





Thank you!